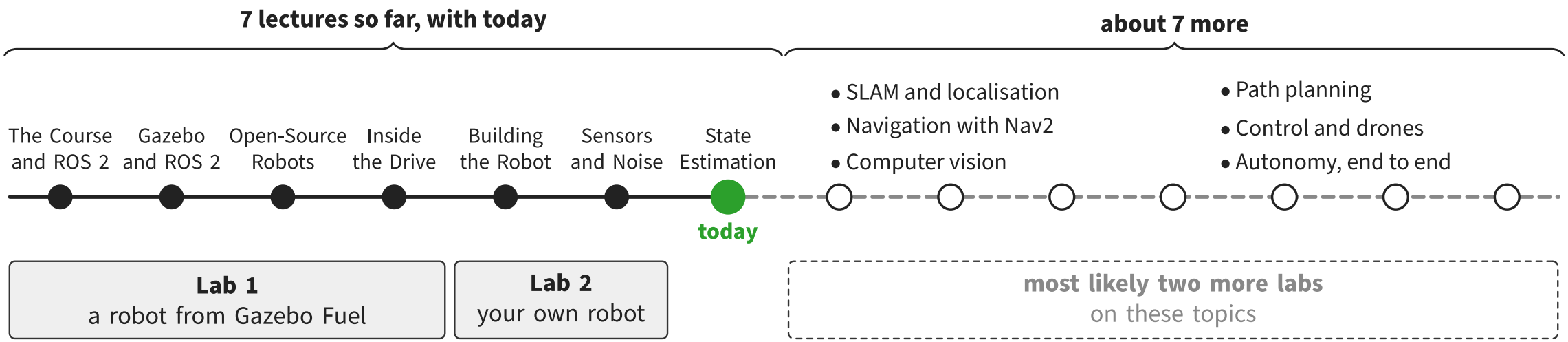
AUTONOMOUS
INTELLIGENCE GROUP

LECTURE 6 — STATE ESTIMATION AND FUSION

ONE ESTIMATE FROM NOISY SENSORS: THE KALMAN FILTER BY HAND AND IN ROS 2

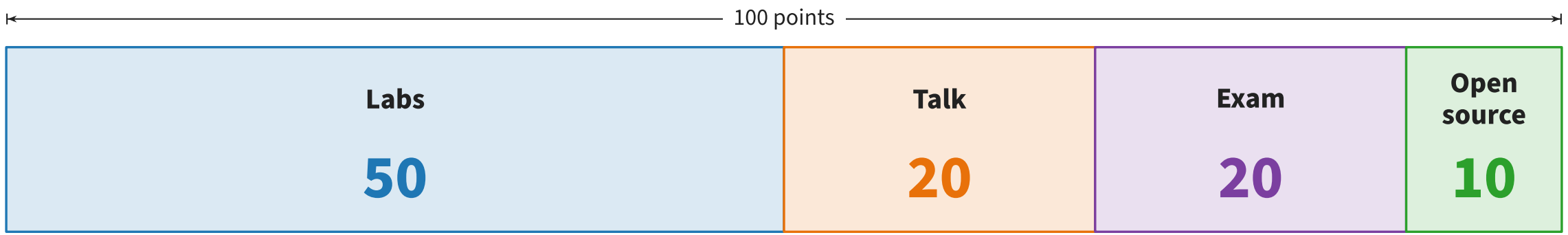
Topics: Course feedback · Lab 2 · Complementary filter · Bayes filter · Kalman filter

HALFWAY THROUGH THE COURSE



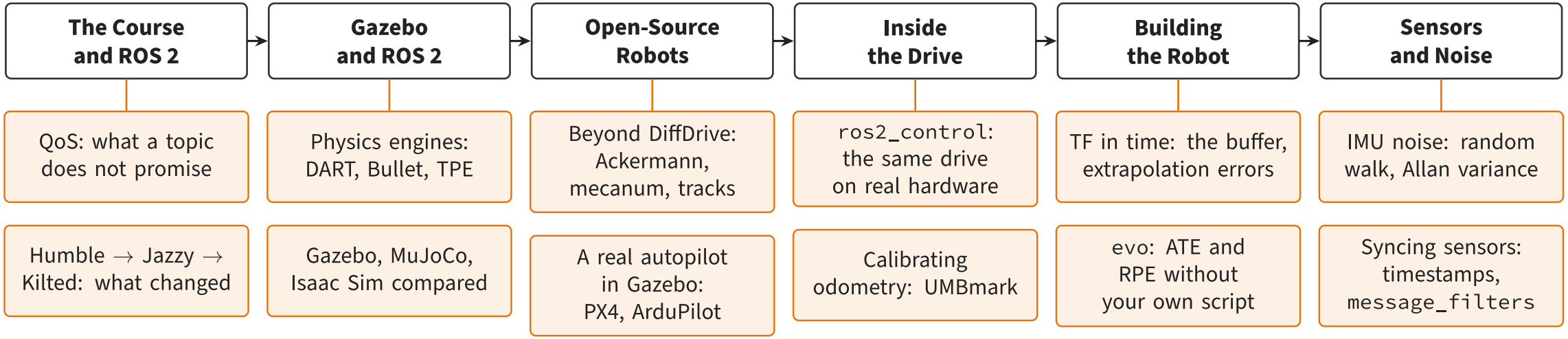
- **Lab 1** — a ready-made robot from Gazebo Fuel, driven from ROS 2: commands in, odometry out, its path plotted
- **Lab 2** — your own robot in URDF with a lidar and an IMU: noise from a real datasheet, checked against ground truth
- **Ahead** — about 7 more lectures on these topics, in rough order; most likely two more labs on them

GRADING: A REMINDER



- **Labs** — the README is what you hand in: how to run it, what you see, your numbers
- **Talk** — 15–20 min on a topic of your choice, agreed with me
- **Exam** — a 15–20 min interview: your results, your decisions and where they stop working
- **Open source** — a repository and its issues, reviewed: where you could contribute

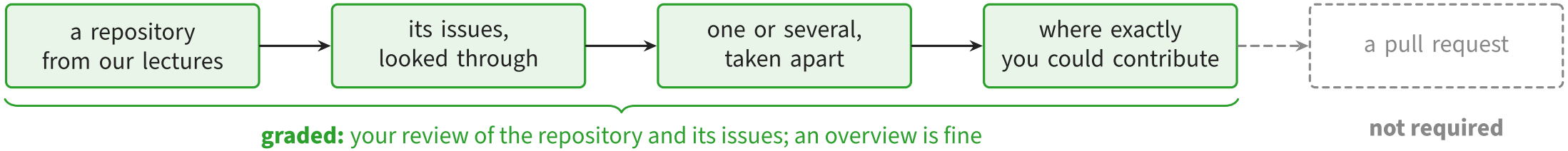
TALK IDEAS: TOPICS WE ONLY TOUCHED



15–20 min, your topic, agreed with me. What stayed unclear in your `notes.txt` can be a topic too.

📖 roscon.ros.org · 📖 docs.ros.org/en/rolling/Releases.html

OPEN SOURCE: REPOSITORIES WE USED



LECTURE	🎯 PUBLIC REPOSITORIES WE USED
The Course and ROS 2	github.com/ros2/common_interfaces · github.com/ROBOTIS-GIT/turtlebot3
Gazebo and ROS 2	github.com/gazebosim/gz-physics · github.com/ros2/rviz
Open-Source Robots	github.com/gazebosim/ros_gz · github.com/ros2/teleop_twist_keyboard
Inside the Drive	github.com/gazebosim/gz-sim · github.com/clearpathrobotics/clearpath_common
Building the Robot	github.com/ros/robot_state_publisher · github.com/MichaelGrupp/evo
Sensors and Noise	github.com/SteveMacenski/slam_toolbox · github.com/MOGI-ROS/Week-5-6-Gazebo-sensors
State Estimation	github.com/cra-ros-pkg/robot_localization · github.com/CCNYRoboticsLab/imu_tools

TODAY: THE COMPLEMENTARY FILTER, BAYES, KALMAN, THE EKF

1 · The Complementary Filter → **2 · Noise and the Average** → **3 · The Bayes Filter: Predict and Update** → **4 · The Kalman Filter in 1D**

Two poor sensors, one good angle: mostly the gyroscope, a little of the accelerometer

A guess and its doubt: averaging shrinks the doubt, but lags when the robot moves

How a scan changes what the robot believes: motion blurs it, a scan sharpens it

The robot drives to the wall: predict with the wheels, update with the lidar, the best weight at every step

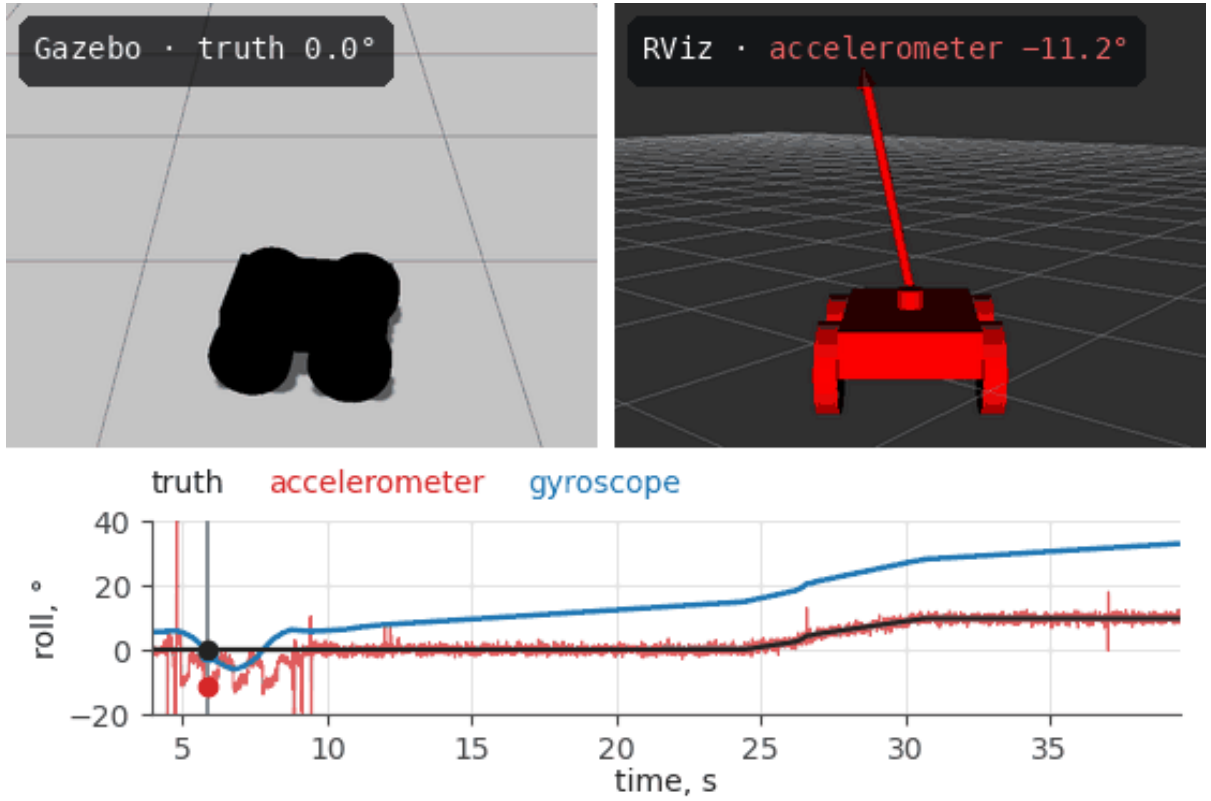
5 · The State and Its Covariance → **6 · When the Robot Turns: the EKF** → **7 · Fusing Wheels and Gyroscope**

A robot on the floor: position and heading, and errors that go together

A straight-line model loses a turning robot; the extended Kalman filter follows the curve

Our robot in ROS 2: the wheels slip in turns, the gyroscope drifts, one filter fuses both

THE GYROSCOPE DRIFTS, THE ACCELEROMETER SHAKES



- **The gyroscope** is smooth, but its small error adds up: it drifts
- **The accelerometer** does not drift, but it shakes, and in a turn it shows a tilt that is not there
- **Together:** each is good where the other is poor — so we fuse them

LEGEND

tilt — how far the robot leans: the angle between its up axis and the vertical

roll — the robot's tilt to the side, °

black line — the true roll, from Gazebo

blue line — the roll from the gyroscope

red line — the roll from the accelerometer

grey vertical line — the moment the pictures show

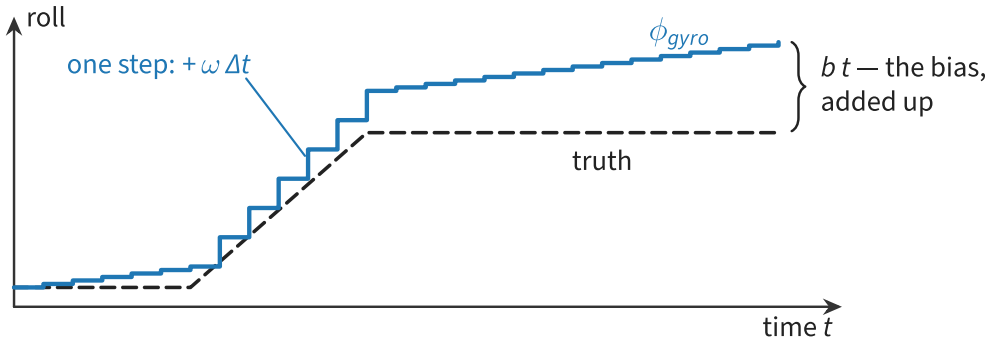
red arrow in RViz — what the accelerometer feels; RViz holds the robot model level

dark boxes — the roll at this moment: the truth in Gazebo, the accelerometer's in RViz

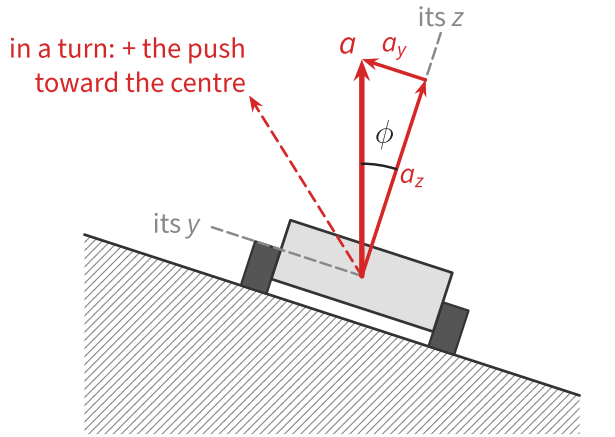
📖 porabook.com/book/

HOW EACH SENSOR MEASURES THE ROLL

gyroscope: add up its rate, step by step



accelerometer: the tilt of gravity's push



standing still, it feels only gravity's push *a*, straight up in the world

split along the robot's axes: the angle between *a* and its *z* is the roll ϕ_{acc}

s3.amazonaws.com/scolton-www/docs/filter.pdf

Each sensor's own roll — the ingredients of the filter:

Gyroscope — its rate, added up step by step:

$$\phi_{gyro} = \Sigma\, \omega\, \Delta t$$

smooth, but it **drifts**: its bias b adds up too, $b\,t$ after a time t

Accelerometer — the tilt of gravity's push:

$$\phi_{acc} = \operatorname{atan2}(a_y, a_z)$$

no drift, but it **shakes**, and in a turn it "tilts" sideways without tilting

LEGEND

ϕ_{gyro}, ϕ_{acc} — the roll from the gyroscope and from the accelerometer, rad

ω — the turn rate from the gyroscope, rad/s

Δt — one IMU step: 0.01 s at 100 Hz

b — the gyroscope's bias: the same small error in every reading

t — the time since the start, s

a_y, a_z — the accelerometer along the robot's own axes: sideways and up, m/s²

$\operatorname{atan2}$ — the angle of a vector from its two parts, with the right sign

blue steps — the gyroscope's sum; dashed — the truth

red arrows — gravity's push a and its two parts

THE GYROSCOPE DRIFTS, THE ACCELEROMETER SHAKES.
HOW DO WE GET ONE GOOD ANGLE FROM THE TWO?

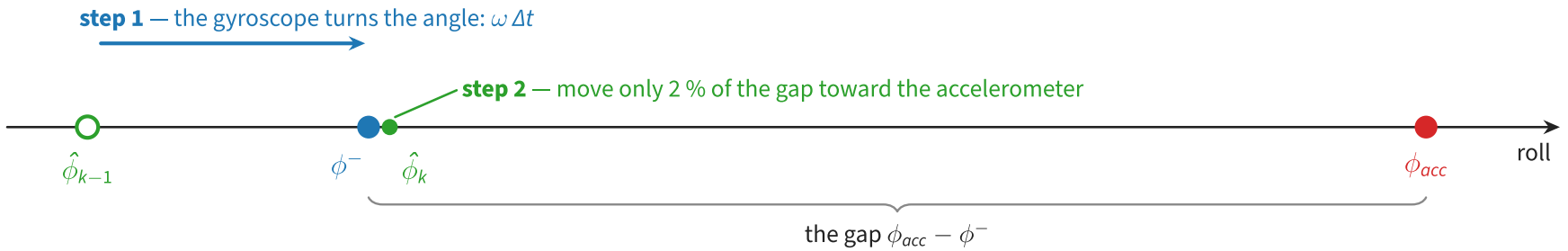
ANSWER: THE COMPLEMENTARY FILTER — 98 % GYROSCOPE, 2 % ACCELEROMETER

Step 1 — predict with the gyroscope: $\phi^- = \hat{\phi}_{k-1} + \omega\,\Delta t$

Step 2 — pull 2 % toward the accelerometer:

$$\hat{\phi}_k = \alpha\,\phi^- + (1-\alpha)\,\phi_{acc}, \qquad \alpha = 0.98$$

The gyroscope's drift is pulled back a little every step; the accelerometer's shaking barely gets in.



youtu.be/whSw42XddsU
ahrs.readthedocs.io/en/latest/filters/complementary.html

LEGEND

ϕ^- — the prediction: the old estimate turned by the gyroscope; the minus — before the measurement corrects it, the usual mark of a prediction

$\hat{\phi}_{k-1}$ — the old estimate; the hat — an estimate, not the truth;

k — the step number

$\omega\,\Delta t$ — the gyroscope's turn in one step: rate ω × one IMU step Δt, 0.01 s

ϕ_{acc} — the accelerometer's roll: the angle of gravity's push from its two parts, $\operatorname{atan2}(a_y, a_z)$

a_y, a_z — the accelerometer along the robot's own axes: sideways and up, m/s²

$\hat{\phi}_k$ — the new estimate

the gap $\phi_{acc} - \phi^-$ — how far the accelerometer is from the prediction

α — the weight of the prediction, between 0 and 1;

$1 - \alpha$ — the accelerometer's

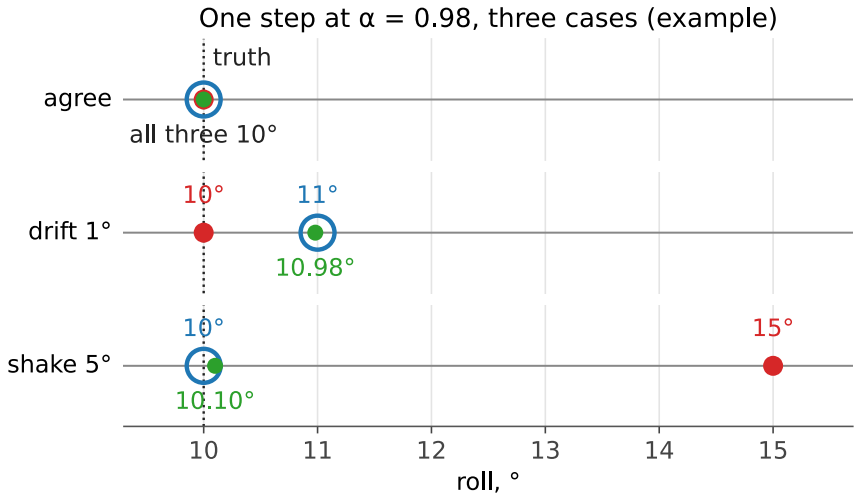
THE GAP IN THREE CASES: AGREE, DRIFT, SHAKE

The same formula, written with the gap between the two angles:

$$\hat{\phi}_k = \phi^- + 0.02\,(\phi_{acc} - \phi^-)$$

The robot stands still on the slope; the true roll is 10° (example):

CASE	ϕ^-	ϕ_{acc}	THE GAP	$\hat{\phi}_k$
agree	10°	10°	0	10 + 0 = **10°**
drift 1°	11°	10°	-1°	11 - 0.02 = **10.98°**
shake 5°	10°	15°	+5°	10 + 0.1 = **10.1°**



LEGEND

blue ring — ϕ^-, the gyroscope's prediction: the old estimate plus one gyroscope step, not its whole sum

red dot — ϕ_{acc}, the accelerometer's roll: the angle of gravity's push from its two parts, $\operatorname{atan2}(a_y, a_z)$

a_y, a_z — the accelerometer along the robot's own axes: sideways and up, m/s²

green dot — $\hat{\phi}_k$, the new estimate; k — the step number

dotted line — the truth, 10° in all three cases

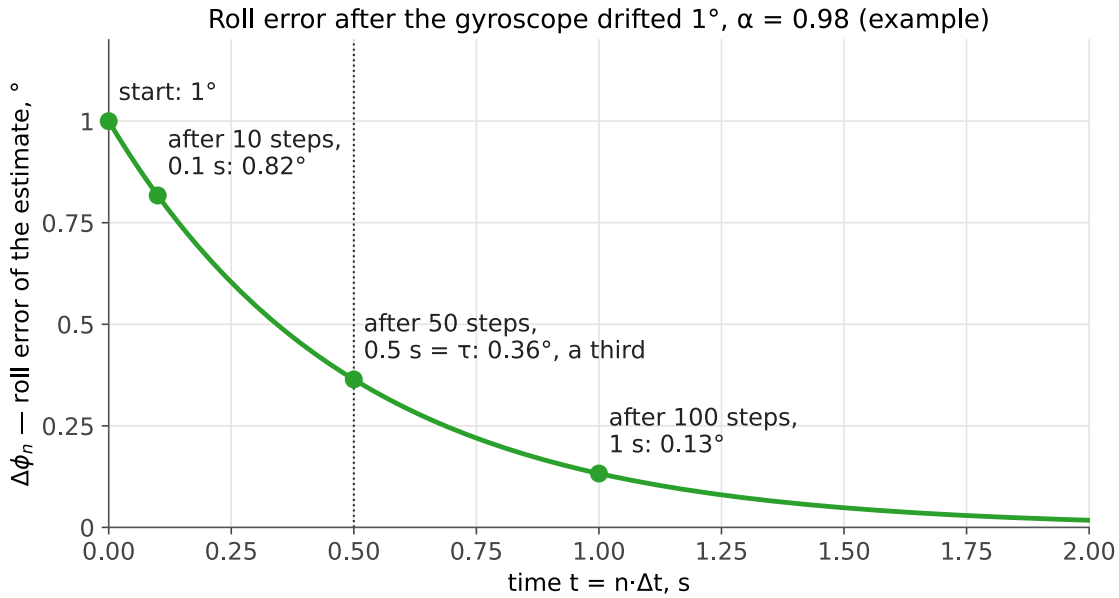
the gap $\phi_{acc} - \phi^-$ — how far the accelerometer is from the prediction

0.02 — the accelerometer's share, $1 - \alpha$, with $\alpha = 0.98$

- **Agree** — no gap, nothing changes: the weights add up to 1
- **Drift** — the gyroscope is 1° off; one step takes 2 % of it away: 0.02°
- **Shake** — the accelerometer jumps 5°; only 2 % of it gets in: 0.1°

s3.amazonaws.com/scolton-www/docs/filter.pdf

THE TIME CONSTANT: HOW LONG AN OLD ERROR LASTS



Each step, the filter:

$$\hat{\phi}_k = \alpha\,\phi^{-} + (1-\alpha)\,\phi_{acc}$$

An old error after n steps:

$$\Delta\phi_n = \alpha^n\,\Delta\phi_0$$

The time constant:

$$\tau \approx \frac{\Delta t}{1-\alpha} = \frac{0.01\ \text{s}}{0.02} = 0.5\ \text{s}$$

How long the filter remembers an old error: after τ a third of it is left.

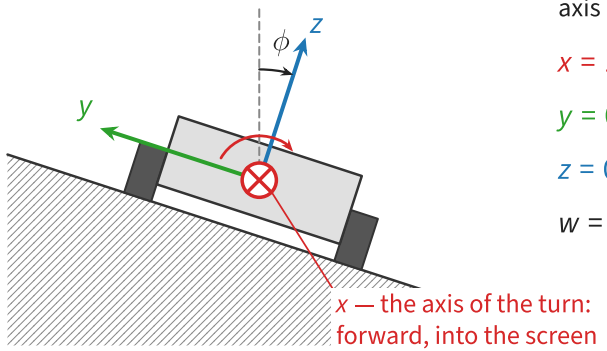
LEGEND

green curve — $\Delta\phi_n$ over time; dots — the start and after 10, 50, 100 steps
dotted line — the moment $t = \tau$
$\Delta\phi_n$ — the roll error after n steps: the estimate minus the true roll, °; $\Delta\phi_0$ — at the start, 1° here
$\hat{\phi}_k$ — the new estimate; k — the step number
ϕ^{-} — the gyroscope's prediction
ϕ_{acc} — the accelerometer's roll
α — the prediction's weight, 0.98 here; $1-\alpha$ — the accelerometer's share
n — the number of steps
τ — the time constant, s
Δt — one IMU step: 0.01 s at 100 Hz

- Only the prediction ϕ^{-} carries the error, weight α: 1° → 0.98° → 0.96° … (example)
- A smaller share of the accelerometer, a longer memory: $1-\alpha$ = 0.01 gives about 1 s
- Changes slower than τ come from the accelerometer, faster ones from the gyroscope

s3.amazonaws.com/scolton-www/docs/filter.pdf

A QUATERNION: ONE ROTATION IN FOUR NUMBERS



this turn as a quaternion, axis (1, 0, 0), angle ϕ:

$x = 1 \cdot \sin\frac{\phi}{2}$

$y = 0 \cdot \sin\frac{\phi}{2} = 0$

$z = 0 \cdot \sin\frac{\phi}{2} = 0$

$w = \cos\frac{\phi}{2}$

- **Any orientation is one turn** about one axis, set by the turn itself: the line it leaves in place — x for a roll, z for a yaw
- **A quaternion** keeps that axis and angle in four numbers, as in `Imu`, `/odom`, `tf2_echo`:

$$(x,\ y,\ z) = \text{axis} \cdot \sin\frac{\text{angle}}{2} \qquad w = \cos\frac{\text{angle}}{2}$$

One rotation in four numbers, and two turns combine by multiplying.

- **Why not roll, pitch, yaw:** order matters, they jump at ±180°, and at a pitch of 90° roll and yaw merge
- **On our slope the robot rolls about x**, forward: axis $(1,\ 0,\ 0)$, angle ϕ = 9.7°, so $y = z = 0$: x = sin 4.85° = 0.085, w = cos 4.85° = 0.996
- No rotation: $(0, 0, 0, 1)$

LEGEND

axis — the line the turn leaves in place, as three numbers along x, y, z; for the roll $(1, 0, 0)$ — x, forward

angle — how far it turns; here the roll ϕ = 9.7°, from the dashed vertical to the robot's z

x, y, z, w — the four numbers of a quaternion, as ROS names them; x, y, z go with the axes x, y, z

$\sin, \cos$ of half the angle — a quaternion keeps half of it: 4.85°

red ⊗, **green**, **blue** — the robot's axes x (into the screen), y, z, as RViz colours them, and their numbers in the quaternion

📖 docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Quaternion-Fundamentals.html

FORMULA ↔ IMU_TOOLS CODE

$$\hat{\phi}_k = \alpha\,\phi^{-} + (1-\alpha)\,\phi_{acc}$$

- $\hat{\phi}_k$ — the new estimate, k — the step · ϕ^{-} — the gyroscope's prediction · ϕ_{acc} — the accelerometer's roll · α — the prediction's weight
- `getPrediction` — step 1, turn by the gyroscope
- `getAccCorrection` — how far to the accelerometer's "down"; `scaleQuaternion` — keeps only the share `gain` of that turn; `gain` $= 1 - \alpha$ (default `0.01`, so $\alpha = 0.99$)
- `quaternionMultiplication` — the new estimate: the prediction turned by that share
- `w…`, `a…` — the gyroscope's and the accelerometer's readings, `dt` — one IMU step Δt; `q…` — the orientation as a quaternion (four numbers of one rotation): the mix is a turn, the same idea in 3D; `qI` — no turn, $(0, 0, 0, 1)$

C++ · imu_complementary_filter/src/complementary_filter.cpp — the filter inside imu_tools' complementary_filter_node

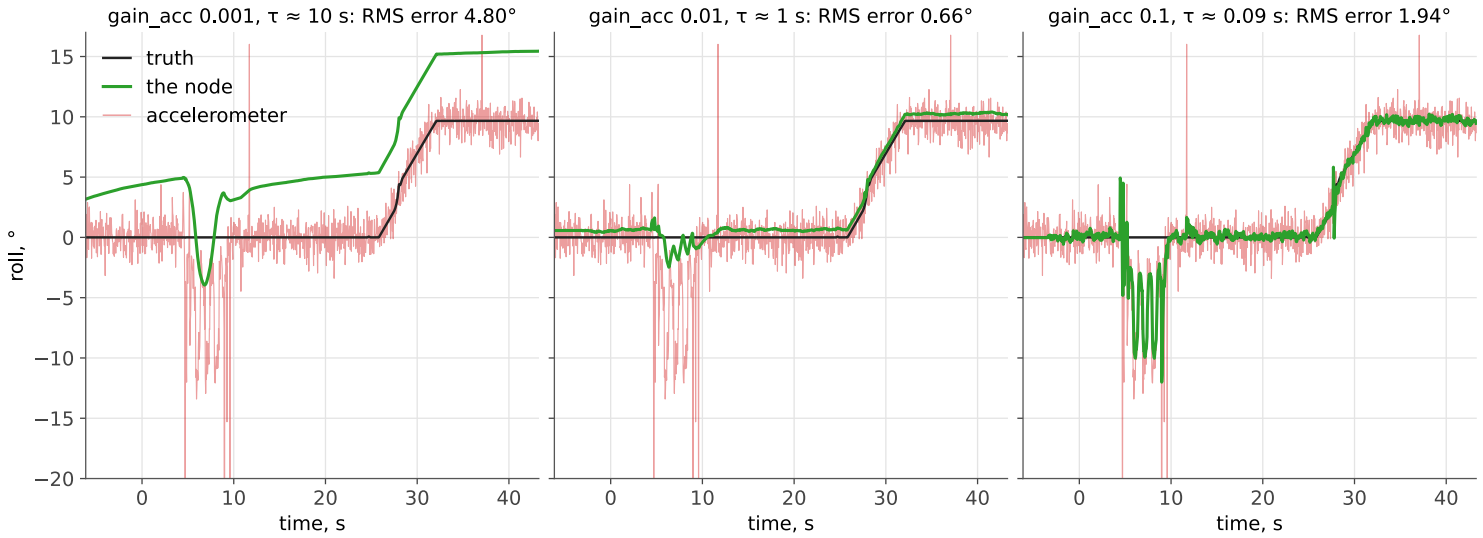
```cpp
// Prediction.
getPrediction(wx, wy, wz, dt, q0_pred, q1_pred, q2_pred, q3_pred);

// Correction (from acc):
// q_ = q_pred * [(1-gain) * qI + gain * dq_acc]
getAccCorrection(ax, ay, az, q0_pred, q1_pred, q2_pred, q3_pred,
                 dq0_acc, dq1_acc, dq2_acc, dq3_acc);
gain = gain_acc_;
scaleQuaternion(gain, dq0_acc, dq1_acc, dq2_acc, dq3_acc);
quaternionMultiplication(q0_pred, q1_pred, q2_pred, q3_pred, dq0_acc,
                         dq1_acc, dq2_acc, dq3_acc, q0_, q1_, q2_, q3_);
```

Parameters: `gain_acc`, `do_adaptive_gain: false`, `do_bias_estimation: false`, `use_mag: false`

🎯 github.com/CCNYRoboticsLab/imu_tools

PICKING THE TIME CONSTANT ON OUR ROBOT: THREE RUNS



$$\hat{\phi}_k = \alpha\,\phi^{-} + (1-\alpha)\,\phi_{acc} \qquad \texttt{gain_acc} = 1-\alpha \qquad \tau \approx \frac{\Delta t}{\texttt{gain_acc}}$$

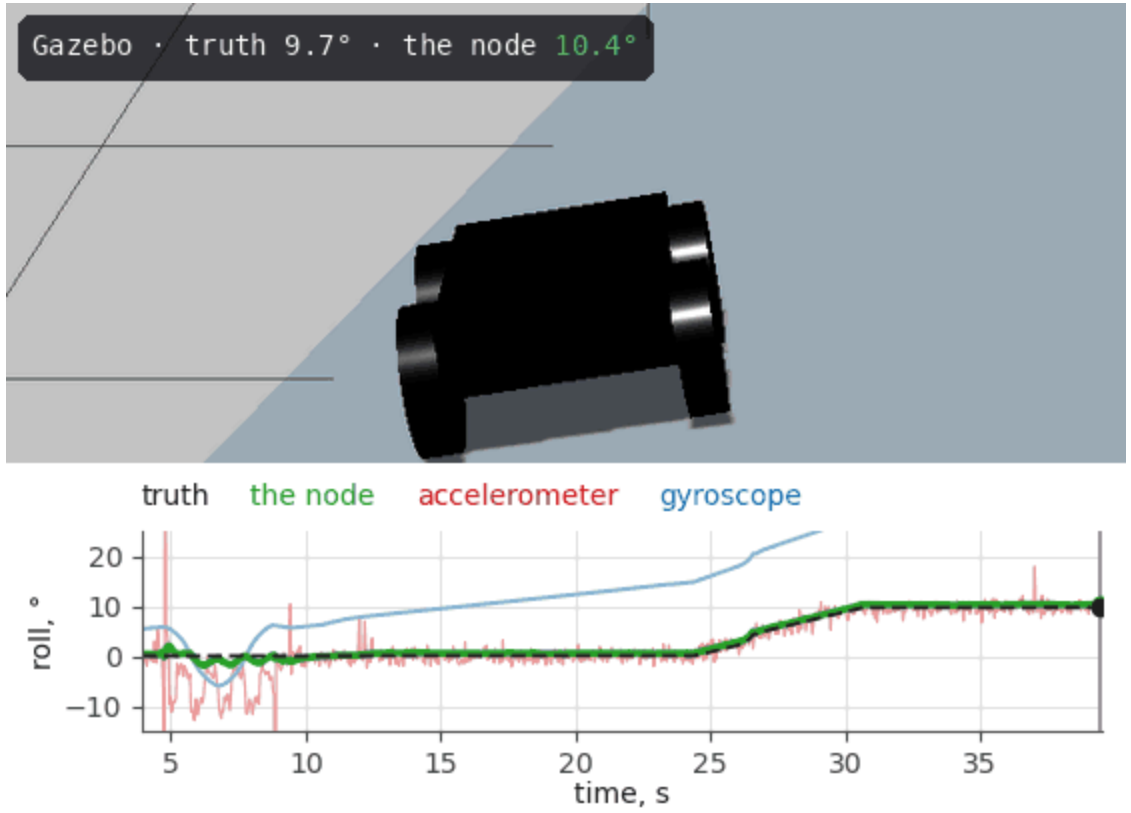
LEGEND

green line — $\hat{\phi}_k$, the node's estimate; k — the step number
light red line — ϕ_{acc}, the accelerometer's roll
black line — the truth
ϕ^{-} — the gyroscope's prediction
α — the prediction's weight
`gain_acc` — imu_tools' name for the accelerometer's share, $1-\alpha$
τ — the time constant, s; Δt — one IMU step, 0.01 s
RMS — root mean square: the typical error over the run

- `complementary_filter_node` on our robot, three runs: `gain_acc` = 0.001, 0.01, 0.1, so $\tau \approx 10$ s, 1 s, 0.09 s; `gain_acc` is ours, set once in the node's parameters
- $\tau \approx 10$ s: smooth, but it keeps the drift · $\tau \approx 0.09$ s: quick, but it believes the false roll in the turn · $\tau \approx 1$ s, the default: the smallest error

🎯 github.com/CCNYRoboticsLab/imu_tools

IMU_TOOLS ON OUR ROBOT: ROLL HOLDS, YAW DRIFTS



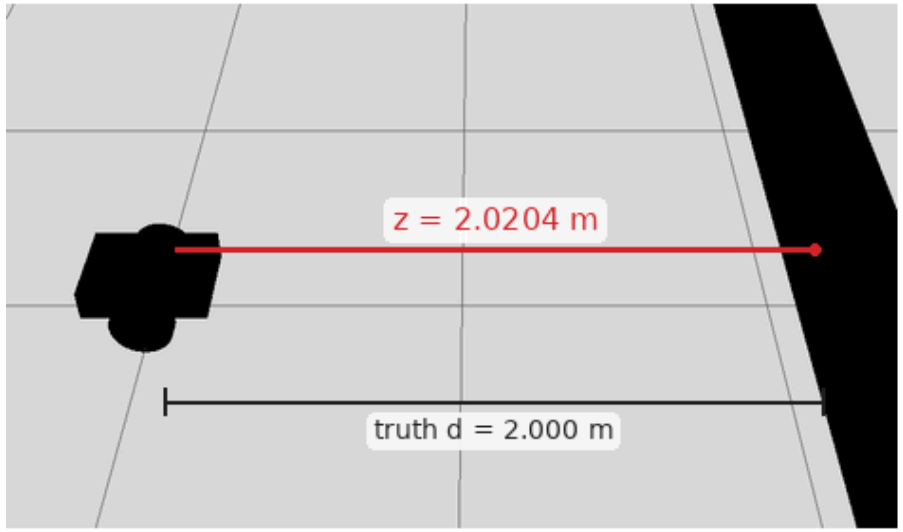
- `/imu/data_raw` → `complementary_filter_node` → `/imu/data`: the same readings, plus an orientation
- `gain_acc` = 0.01, the default: the roll is 0.6° off on average, 2° at worst — in the turn
- standing on the slope it stays 0.6° off: each step the bias b adds $b\,\Delta t$, the accelerometer takes 1 % back; they balance near $b\,\tau \approx 0.6°/s \times 1\ s$
- **yaw still drifts**: gravity says nothing about a turn around the vertical; later: the lidar fixes it
- we picked the weight by trying. **Is there a best weight — and what if the robot moves?**

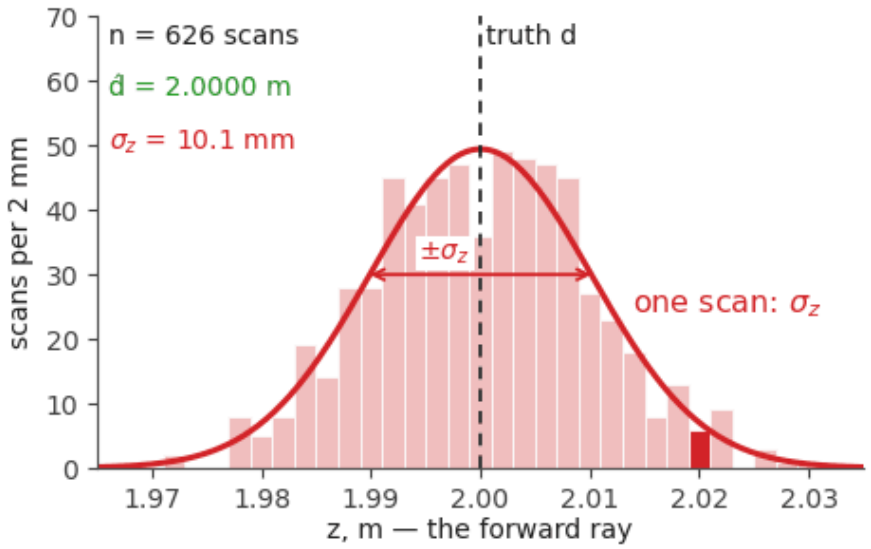
LEGEND

dashed line — the true roll, from Gazebo
green line — the node's roll
faint red — the accelerometer;
faint blue — the gyroscope
grey vertical line — the moment Gazebo shows
b — the gyroscope's bias: the same small error in every reading
Δt — one IMU step, 0.01 s
τ — the time constant: about 1 s at `gain_acc` 0.01
yaw — the turn around the vertical

🎯 github.com/CCNYRoboticsLab/imu_tools

A BELIEF IS A GUESS AND ITS DOUBT





$$\mathcal{N}(\hat{d},\ \sigma^2_{\hat{d}})$$

A belief is a best guess $\hat{d}$ and its typical error $\sigma_{\hat{d}}$: the Gaussian says how likely each distance is.

The red Gaussian is where one scan lands: after one scan $\hat{d} = z, \sigma_{\hat{d}} = \sigma_z$.

LEGEND

$\mathcal{N}(\hat{d}, \sigma^2_{\hat{d}})$ — a normal distribution, a Gaussian: centre $\hat{d}$, variance $\sigma^2_{\hat{d}}$

$\hat{d}$ — the best guess of the distance, m; the hat — an estimate

$\sigma_{\hat{d}}$ — its typical error: the Gaussian's width; σ^2 — a variance

z — one scan of the lidar's forward ray, m

σ_z — how far one scan scatters: 10.1 mm, measured

red bars — the scans, 2 mm bins, the latest bright; **red curve** — where one scan lands

green number — $\hat{d}$, the mean so far; dashed — the truth

probability — the area under the curve; ±σ holds 68 %

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/03-Gaussians.ipynb

AVERAGING SHRINKS THE ERROR

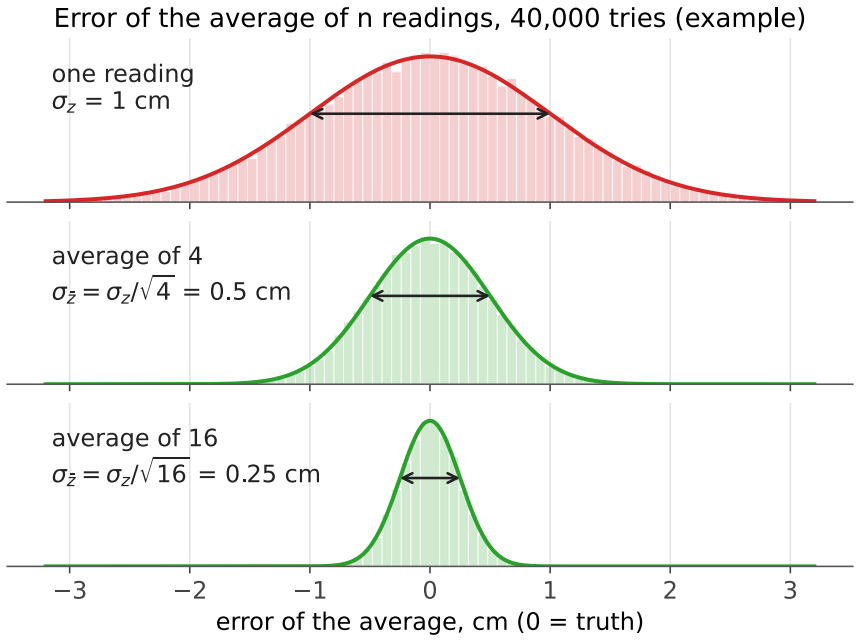
Each of 4 readings is off by +1 or −1 cm, equally likely (example): σ_z = 1 cm

SUM OF ERRORS	WAYS	ERROR OF THE AVERAGE	SQUARED
±4	1 + 1	±1	1
±2	4 + 4	±0.5	0.25
0	6	0	0

mean square = (2·1 + 8·0.25)/16 = **0.25** → $\sigma_{\bar{z}}$ = $\sqrt{0.25}$ = **0.5 cm**

$$\sigma_{\bar{z}} = \frac{\sigma_z}{\sqrt{n}}$$

How many readings to average: 4 halve the error, 16 quarter it.



LEGEND

σ_z — the error of one reading (one scan)

n — the number of readings averaged

$\bar{z}$ — their average; a bar means an average

$\sigma_{\bar{z}}$ — the error of their average: the root of the mean square

16 = 2^4 sign combinations of four readings; 1-4-6-4-1 — how many of them give each sum

red — one reading; **green** — the average of 4 and of 16: bars — 40,000 tries, curves — the Gaussian of the formula, arrows — ±σ

youtu.be/7mYDHbrLEQo?t=141

THE RECURSIVE AVERAGE KEEPS NO OLD SCANS

$$\hat{d}_n = \hat{d}_{n-1} + \frac{1}{n}\left(z_n - \hat{d}_{n-1}\right)$$

The recursive average: the same average, keeping no old scans.

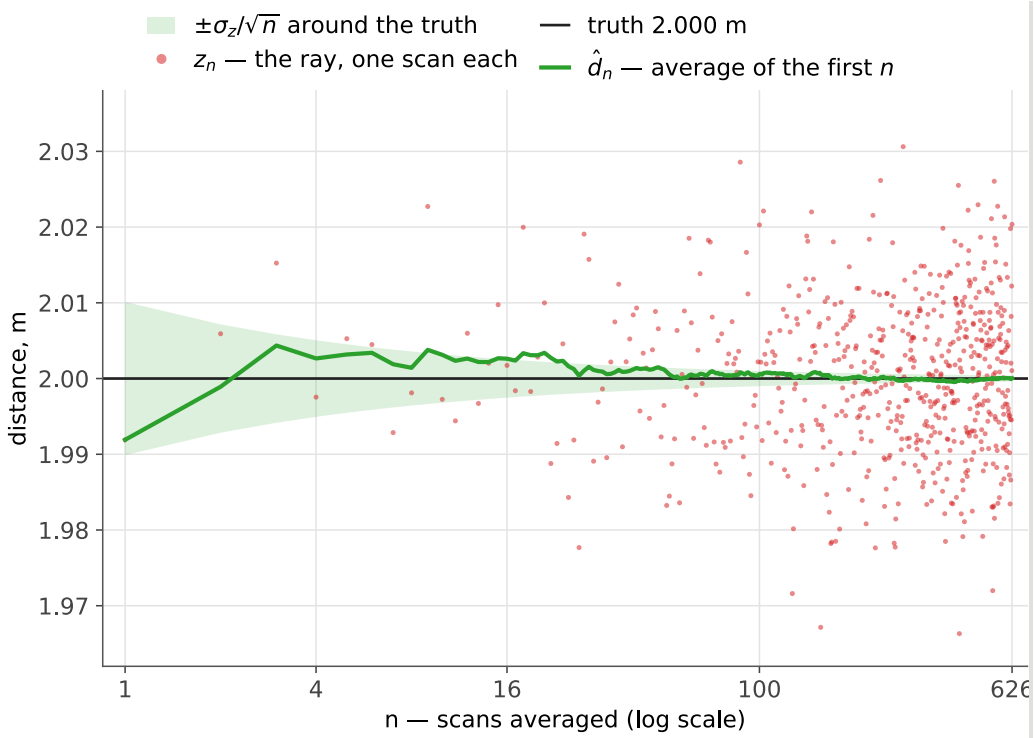
Four scans (example):

n	z_n, M	$\hat{d}_n$, M
1	2.04	2.04
2	2.02	2.04 + ½ · (−0.02) = 2.03
3	2.03	2.03 + ⅓ · 0 = 2.03
4	2.07	2.03 + ¼ · 0.04 = 2.04

Python · pandas — a table library: the mean of all scans so far

```
d_hat = z.expanding().mean()    # running average
```

📖 pandas.pydata.org/docs/reference/api/pandas.Series.expanding.html



LEGEND

n — the number of the scan

z_n — the n-th scan of the lidar's forward ray, m

$\hat{d}_n$ — the average of the first n scans; $\hat{d}_{n-1}$ — before the new one

$\frac{1}{n}$ — the share of the new scan

$z_n - \hat{d}_{n-1}$ — how much the new scan disagrees

same as $(z_1 + \cdots + z_n)/n$: multiply both sides by n

`z` — all scans in a pandas column (`Series`), one per row

`expanding()` — windows that grow by one scan: z_1; z_1, z_2; z_1, z_2, z_3; …

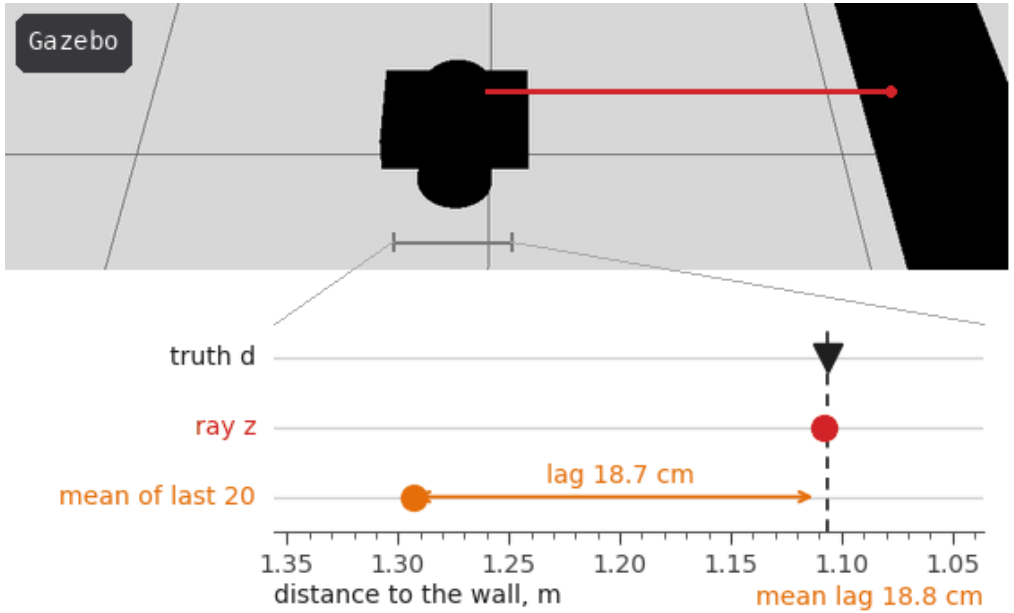
`.mean()` — each window's average: `d_hat` = $\hat{d}_1, \hat{d}_2, \ldots$

red dots — the scans; **green line** — their average; black line — the truth, 2.000 m

light band — ±σ_z/√n: the average's expected error; σ_z — one scan's error, 10.1 mm

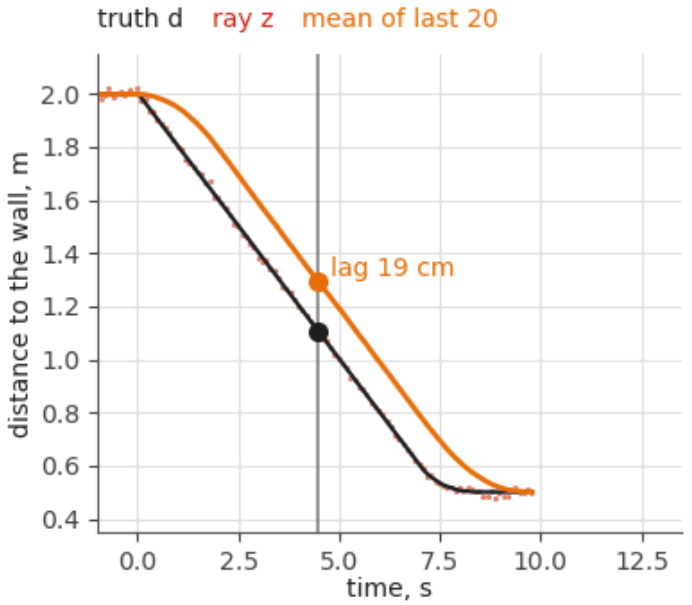
THEN AVERAGE THE LAST 20 SCANS WHILE DRIVING?

ANSWER: NO — A MOVING AVERAGE IS LATE



$$\text{lag} = \frac{n-1}{2}\, v\, \Delta t = 9.5 \cdot 0.2\ \text{m/s} \cdot 0.1\ \text{s} = 19.0\ \text{cm}$$

How far the average of the last n scans stays behind a moving robot; measured: 18.8 cm.



Python · pandas — a table library: the mean of the last 20 scans

```
d_avg = z.rolling(20).mean()
```

LEGEND

lag — how far the average is behind the truth, cm

n — scans in the window, 20

v — speed toward the wall, m/s

Δt — time between scans, 0.1 s

d — the true distance to the wall, z — the lidar's forward ray, m

$\frac{n-1}{2}$ — the window's middle lags this many scans: the mean of 0, 1, …, $n-1$

`z` — all scans in a pandas column (`Series`), one per row

`rolling(20)` — a window of the last 20 scans that slides by one: $z_1 \dots z_{20}$; $z_2 \dots z_{21}$; …

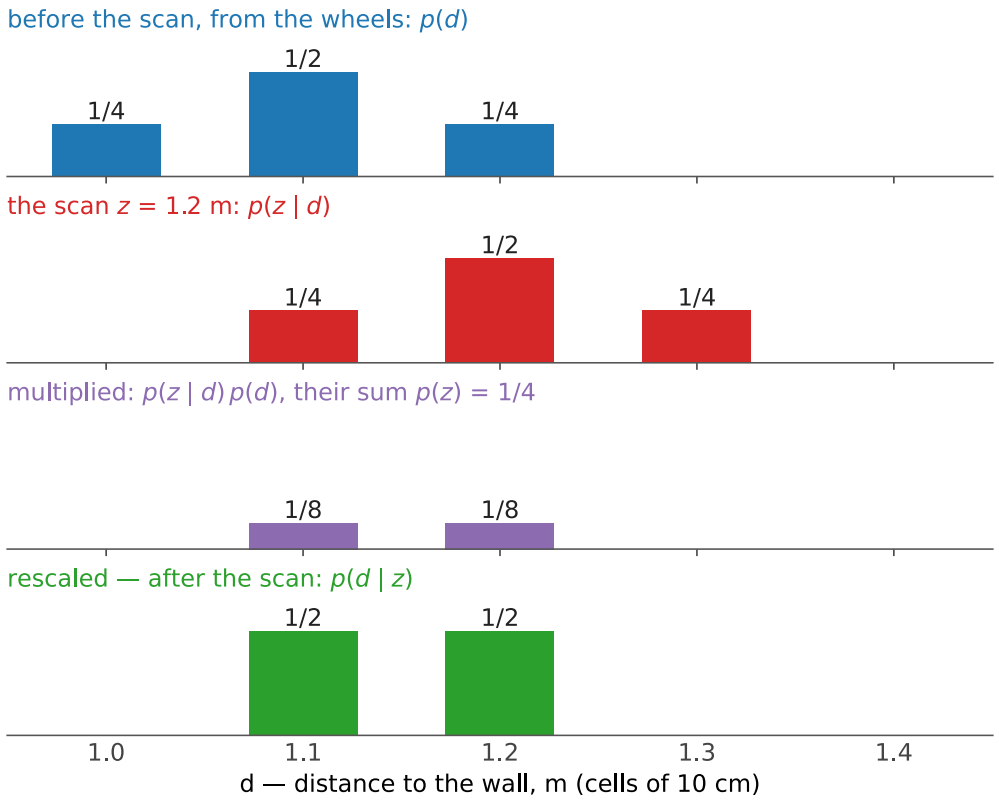
`.mean()` — each window's average: `d_avg`, the orange line

black — the truth · **red** — the ray · **orange** — the mean of the last 20 scans

grey vertical line — the moment the scene shows; dots — the truth and the mean at it

pandas.pydata.org/docs/reference/api/pandas.Series.rolling.html

BAYES' RULE: THE BELIEF AFTER A SCAN



Our robot before the wall, five cells of 10 cm (example):

- **The robot knows** its belief before the scan — the wheels say about 1.1 m; how its lidar errs — by a cell at most; and the scan — the forward ray reads 1.2 m
- **It does not know** the true distance d
- **It computes** the belief after the scan, for every cell

$$p(d \mid z) = \frac{p(z \mid d)\, p(d)}{p(z)}$$

Bayes' rule: from "how likely this scan is at each distance" to "how likely each distance is, given this scan".

LEGEND

d — a cell: a distance to the wall, m

z — the scan: what the lidar's forward ray reads, m

$p(\cdot)$ — a probability, 0 to 1; the bar $\mid$ reads «given»

$p(d)$ — the belief before the scan (prior): **blue bars**

$p(z \mid d)$ — the likelihood: how likely the ray reads z if the wall is at d: **red bars**

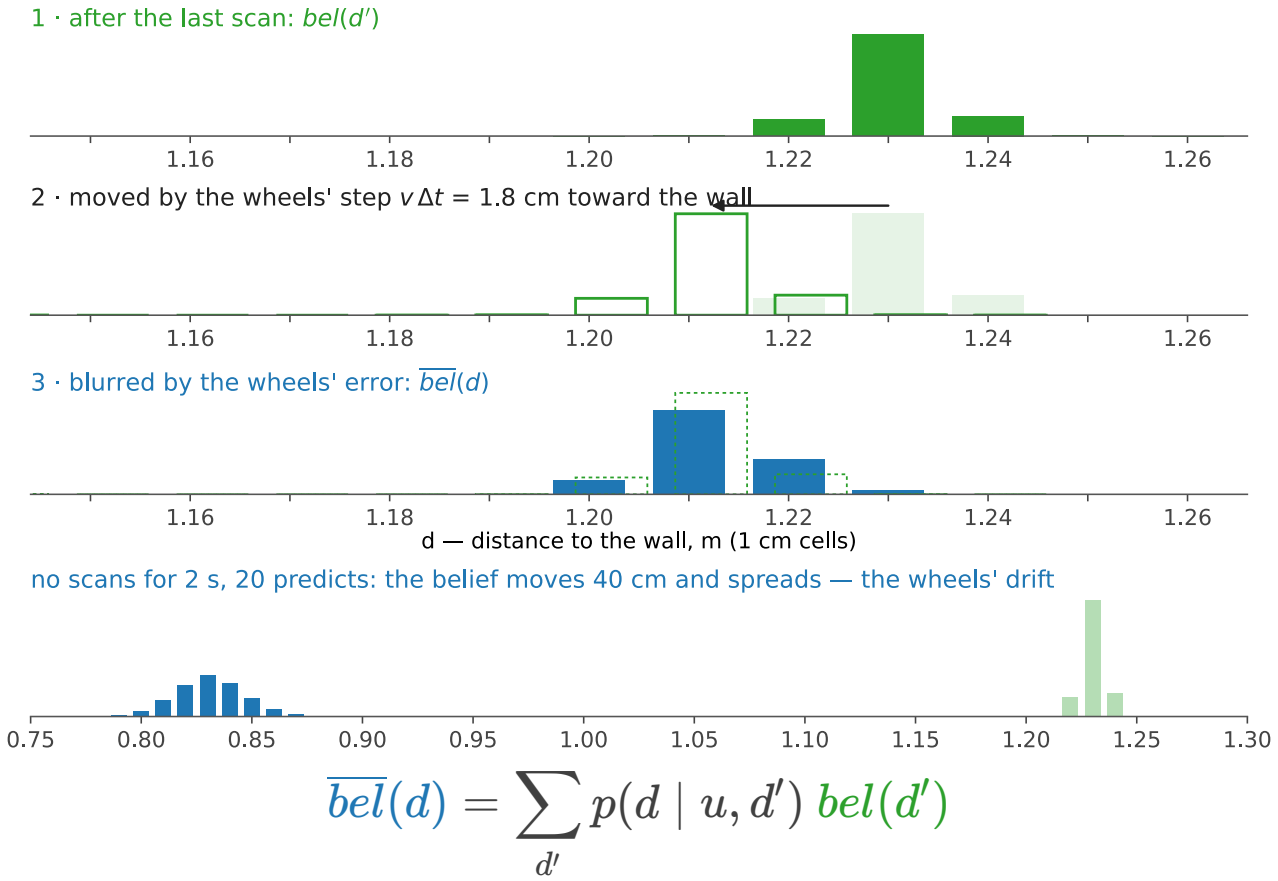
purple bars — the products, cell by cell

$p(z)$ — their sum; divided by it, the bars add up to 1

$p(d \mid z)$ — the belief after the scan (posterior): **green bars**

www.3blue1brown.com/lessons/bayes-theorem

PREDICT ON OUR GRID: MOVE THE BELIEF AND BLUR IT



$$\overline{bel}(d) = \sum_{d'} p(d \mid u, d')\, bel(d')$$

Predict: each cell collects what the motion brings into it.

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/02-Discrete-Bayes.ipynb

LEGEND

d — a cell after the motion: a distance to the wall, m

d' — a cell before it; the sum runs over all of them

u — the motion: the wheels' step $v\,\Delta t$, here 1.8 cm

$bel(d')$ — the belief after the last scan: **green bars**

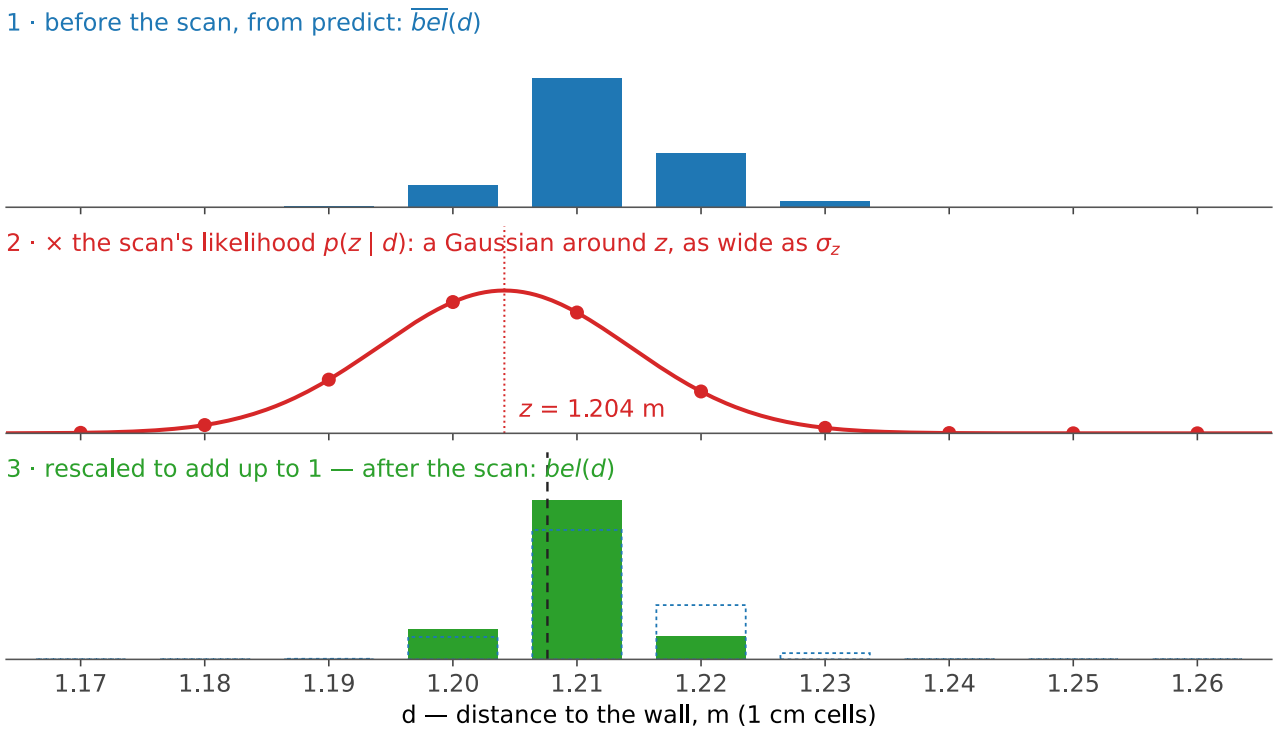
$p(d \mid u, d')$ — the chance that the step u takes the robot from d' to d: the wheels err

$\overline{bel}(d)$ — the belief before the next scan: **blue bars**; the bar on top — before the scan

outlined bars — the green bars, moved by u: each blue bar collects from those around it

the belief — one number per cell; the robot keeps nothing else

UPDATE ON OUR GRID: MULTIPLY BY THE SCAN'S LIKELIHOOD



$$bel(d) = \eta\, p(z \mid d)\, \overline{bel}(d)$$

LEGEND

d — a cell: a distance to the wall, m

z — the scan: what the lidar's forward ray reads, m

$\overline{bel}(d)$ — the belief before the scan, from predict: **blue bars**

$p(z \mid d)$ — the likelihood: how likely the scan z is if the wall is at d; a Gaussian around z, as wide as the lidar's error σ_z: **red curve**

$bel(d)$ — the belief after the scan: **green bars**; outlined — the blue bars, to compare

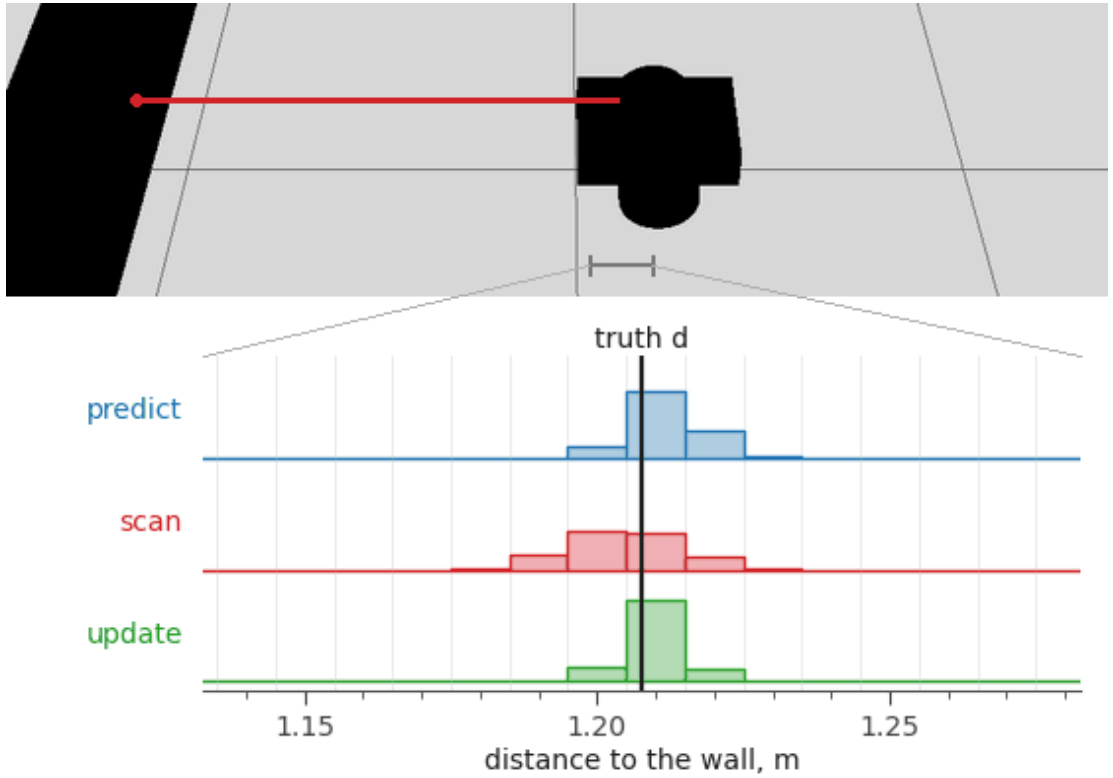
η — 1 / the sum of the products, so the bars add up to 1: the $1/p(z)$ of Bayes' rule

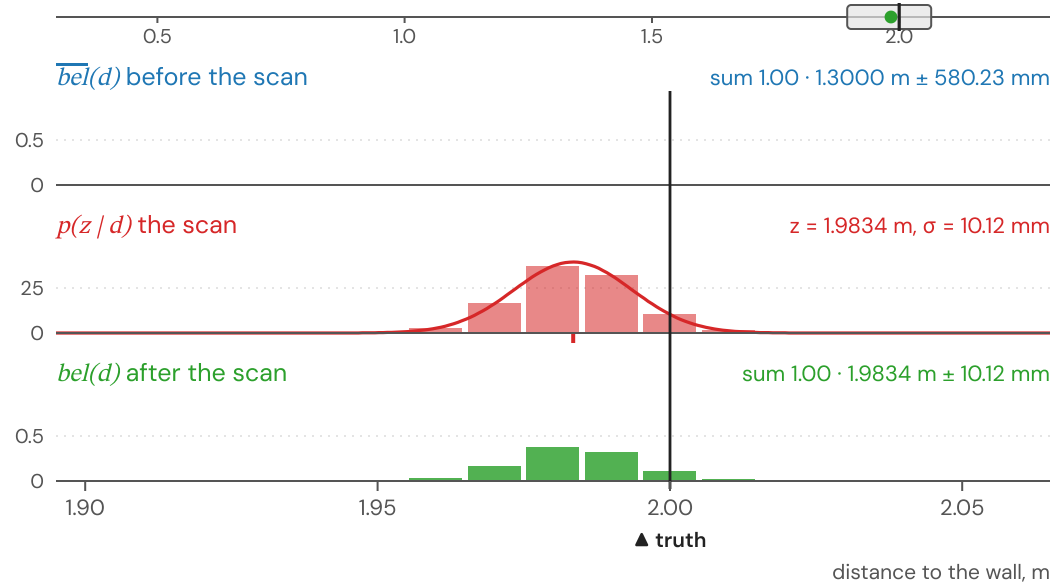
dashed line — the truth

Update: Bayes' rule on every cell — multiply by how well the cell explains the scan, then rescale.

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/02-Discrete-Bayes.ipynb

THE RECIPE ON A GRID: PREDICT, UPDATE, REPEAT





predict | update | ↺ reset | scan 9: still | scan 49 | scan 93: stop

scan 0 of 0–144

t = −0.92 s · update: z = 1.9834 m · after the scan 1.9834 m ± 10.12 mm · truth 2.0000 m

predict — move and blur: $\overline{bel}(d) = \sum_{d'} p(d \mid u, d')\, bel(d')$ · **scan** — its likelihood $p(z \mid d)$ · **update** — multiply, rescale: $bel(d) = \eta\, p(z \mid d)\, \overline{bel}(d)$ · repeat

d, d' — a cell now and before the motion · u — the motion · z — the scan · η — 1 / the sum · σ in the rows — a typical error · t — time from the start

FORMULA ↔ FILTERPY.DISCRETE_BAYES

$$\overline{bel}(d) = \sum_{d'} p(d \mid u, d')\, bel(d')$$

- `np.roll` / `shift` — move every cell by the motion u (`offset`, in cells)
- `convolve(…, kernel)` — the formula's sum: each cell collects its neighbours' belief with the weights `kernel = [0.1, 0.8, 0.1]` (example) — a blur by the wheels' error
- `mode='wrap'` — the cells close into a ring; `'constant'` — a wall: our drive; `cval` — what fills the cells that come in from beyond the wall: 0

$$bel(d) = \eta\, p(z \mid d)\, \overline{bel}(d)$$

- `prior` * `likelihood` — cell by cell
- `normalize` — divide by the sum: this is η; `np.asarray(…, dtype=float)` — the belief as an array of fractions

Python · filterpy/discrete_bayes/discrete_bayes.py — filterpy's grid Bayes filter, library code

```python
def predict(pdf, offset, kernel, mode='wrap', cval=0.):
    if mode == 'wrap':
        return convolve(np.roll(pdf, offset), kernel, mode='wrap')

    return convolve(shift(pdf, offset, cval=cval), kernel,
                    cval=cval, mode='constant')

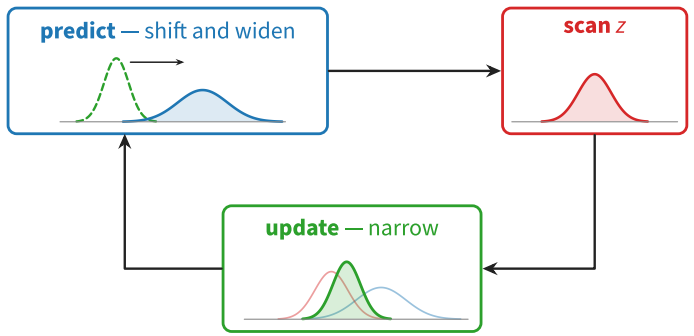
def update(likelihood, prior):
    posterior = prior * likelihood
    return normalize(posterior)

def normalize(pdf):
    pdf /= sum(np.asarray(pdf, dtype=float))
    return pdf
```

- d, d' — a cell now and before the motion; u — the motion; `pdf` — the belief, one number per cell
- $\overline{bel}, bel$ — the belief before and after the scan: `prior`, `posterior`
- $p(z \mid d)$ — the scan's likelihood over the cells: `likelihood`; η — 1 / sum

🎯 github.com/rlabbe/filterpy
▶ colab.research.google.com/github/StanfordASL/pora-exercises/blob/main/ch11/discrete_bayes.ipynb

THE KALMAN FILTER: PREDICT, THEN UPDATE



predict — the wheels move the guess; the doubt grows

$$d^- = \hat{d} - v\,\Delta t \qquad \sigma_{d^-}^2 = \sigma_{\hat{d}}^2 + \sigma_u^2$$

update — the scan pulls the guess; the doubt shrinks

$$K = \frac{\sigma_{d^-}^2}{\sigma_{d^-}^2 + \sigma_z^2}$$

$$\hat{d} = d^- + K\,(z - d^-) \qquad \sigma_{\hat{d}}^2 = (1 - K)\,\sigma_{d^-}^2$$

The grid's recipe for one Gaussian: two numbers per step — the guess and its doubt.

	BAYES FILTER ON A GRID	KALMAN FILTER
the belief	one number per cell: 201 numbers	one Gaussian: $\hat{d}$ and $\sigma_{\hat{d}}$
predict	move and blur every cell	move the guess, add the variances
update	multiply every cell by the likelihood, rescale	a weighted mean, the weight K
it needs	nothing: any shape, even two peaks	a belief that stays one Gaussian

LEGEND

$\hat{d}$ — the estimate of the distance to the wall; the hat — an estimate

$\sigma_{\hat{d}}$ — its typical error; σ^2 — its variance, the doubt

d^-, σ_{d^-} — the prediction and its error; the minus — before the scan

$v\,\Delta t$ — one step from the wheels: speed × time

σ_u — the error one step adds

z — the scan of the lidar's forward ray; σ_z — its error

K — the Kalman gain: the scan's weight, from 0 to 1

$z - d^-$ — the innovation: how far the scan is from the prediction

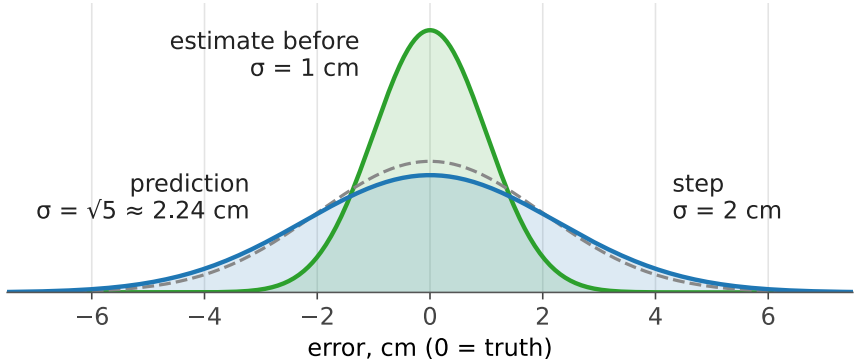
youtu.be/ul3u2yLPwU0

KALMAN PREDICT: THE DOUBT GROWS

The estimate is off by ±1 cm; one step of the wheels adds ±2 cm (example):

ESTIMATE	STEP	PREDICTION	SQUARED
+1	+2	+3	9
+1	−2	−1	1
−1	+2	+1	1
−1	−2	−3	9

mean square = 20/4 = **5** = 1 + 4 → $\sigma_{d^-} \approx$ **2.24 cm**



$$d^- = \hat{d} - v\,\Delta t \qquad \sigma_{d^-}^2 = \sigma_{\hat{d}}^2 + \sigma_u^2$$

The Gaussian moves with the robot and widens: variances add.

Python · filterpy/stats/stats.py — filterpy's helper that adds two Gaussians, library code

```
def add(mean1, var1, mean2, var2):
    return (mean1+mean2, var1+var2)
```

LEGEND

$\hat{d}$ — the estimate before the step; $\sigma_{\hat{d}}$ — its typical error, 1 cm (example)

$v\,\Delta t$ — the step, from the wheels

σ_u — the error one step adds: the wheels', 2 cm (example)

d^-, σ_{d^-} — the prediction and its error; the minus — before the scan

σ^2 — a variance; variances add, σ do not: 1 + 4 = 5, so σ_{d^-} = √5, not 1 + 2

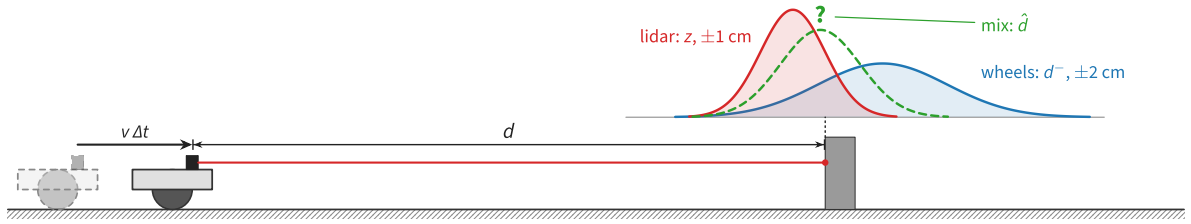
`mean1`, `var1` — the estimate: $\hat{d}$, $\sigma_{\hat{d}}^2$; `mean2`, `var2` — the step: $-v\,\Delta t$, σ_u^2

green — the estimate; dashed — the step's error; blue — the prediction

Only a measurement shrinks the doubt — the update, next.

youtu.be/d_qvLDhkg00

WHEELS AND LIDAR: WHICH WEIGHT IS BEST?



- **The wheels predict:** $d^- = \hat{d}_{k-1} - v\,\Delta t$, off by σ_{d^-} = 2 cm (example)
- **The lidar measures** with its forward ray, one of 360, straight at the wall: $z = d + \text{noise}$, off by σ_z = 1 cm (example)
- **Mix them** with a weight w for the scan:

$$\hat{d} = (1 - w)\, d^- + w\, z$$

The same weighted mix as in the complementary filter: which w is best?

	COMPLEMENTARY FILTER	KALMAN FILTER
the weight	fixed, picked by eye: 0.02	computed at every step from the two variances
the doubt	not kept	kept as σ^2: how sure the estimate is
what it estimates	one angle	any state: a distance here, a pose later

LEGEND

d — the true distance to the wall

d^- — the prediction from the wheels; the minus — before the scan

σ_{d^-} — the prediction's typical error

$\hat{d}_{k-1}$ — the estimate one step earlier, at step $k-1$

z — the lidar's forward ray; σ_z — its typical error

$\hat{d}$ — the mix: the new estimate

$v\,\Delta t$ — how far the robot moved in one step: speed × time

w — the scan's weight, from 0 to 1

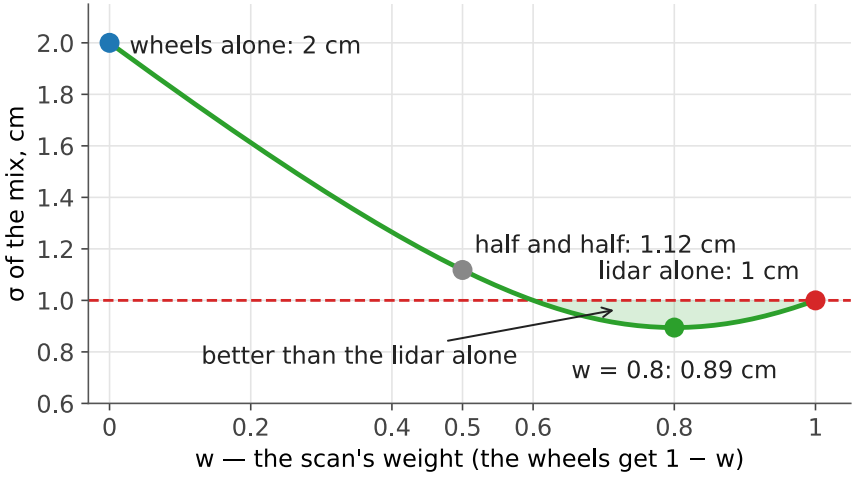
blue, **red** — where the wall is by the wheels and by the lidar; **dashed green** — the mix, not known yet

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/04-One-Dimensional-Kalman-Filters.ipynb

THE RIGHT WEIGHT BEATS BOTH SENSORS

Each error is + or −, equally likely: lidar ±1 cm, wheels ±2 cm (example). Mix with $w = 0.8$:

LIDAR	WHEELS	MIX	SQUARED
+1	+2	0.8 + 0.4 = 1.2	1.44
+1	−2	0.8 − 0.4 = 0.4	0.16
−1	+2	−0.8 + 0.4 = −0.4	0.16
−1	−2	−0.8 − 0.4 = −1.2	1.44



mean square = (1.44 + 0.16 + 0.16 + 1.44)/4 = **0.8** → $\sigma_{\hat{d}} \approx$ **0.89 cm**

half and half, $w = 0.5$: errors ±1.5 and ±0.5 → $\sigma_{\hat{d}} \approx$ **1.12 cm**, worse than the lidar alone

LEGEND

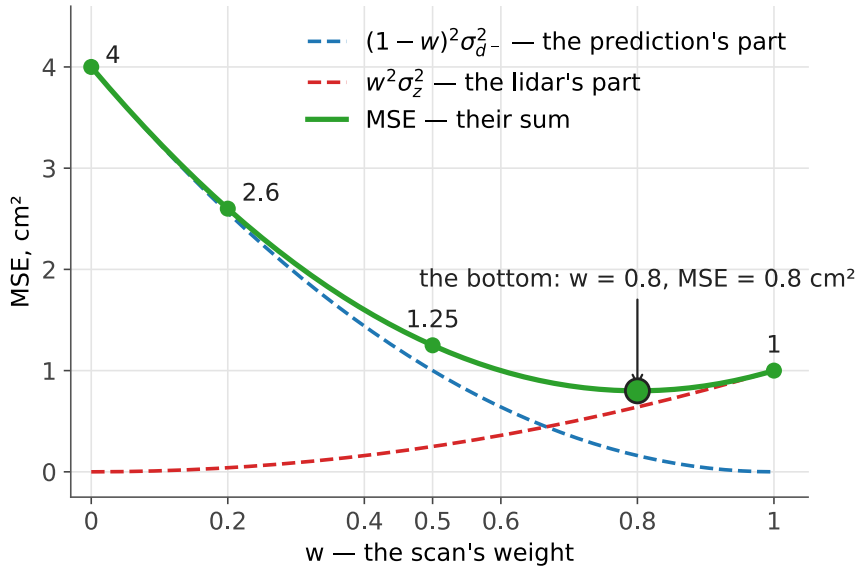
w — the scan's weight: here 0.8 for the lidar, 0.2 for the wheels
lidar, wheels — the error of each, cm; every sign equally likely
mix — 0.8 × the lidar's error + 0.2 × the wheels' error
mean square — the average of the error²
$\sigma_{\hat{d}}$ — the mix's typical error: the root of the mean square
lidar alone: σ_z = 1 cm · wheels alone: σ_{d^-} = 2 cm (example)
green curve — $\sigma_{\hat{d}}$ for every weight; **red dashed** — the lidar alone; shaded — where the mix beats it

Here w = 0.8 is ours, for the example. **The Kalman filter computes the weight itself, at every scan** — from the wheels' and the lidar's errors, which we set once.

THE ERROR OF THE MIX IS A PARABOLA

w	$(1-w)^2\sigma_{d^-}^2$	$w^2\sigma_z^2$	MSE
0	4	0	4
0.2	2.56	0.04	2.6
0.5	1	0.25	1.25
0.8	0.16	0.64	**0.8**
1	0	1	1

$\sigma_{d^-}^2 = 4, \sigma_z^2 = 1$ cm² (example)



LEGEND

MSE — mean squared error: the average of the error², cm²

w — the scan's weight, from 0 to 1

$\sigma_{d^-}^2$ — the prediction's variance: its typical error, squared

σ_z^2 — the lidar's variance

$(1-w)^2\sigma_{d^-}^2$ — what is left of the wheels' error: **blue dashed**

$w^2\sigma_z^2$ — what the lidar brings in: **red dashed**

green — their sum, the MSE; dots — the table's rows

$$\mathrm{MSE}(w) = (1-w)^2\,\sigma_{d^-}^2 + w^2\,\sigma_z^2$$

The error of the mix for every weight: the sum of the two parts has a bottom.

youtu.be/ul3u2yLPwU0

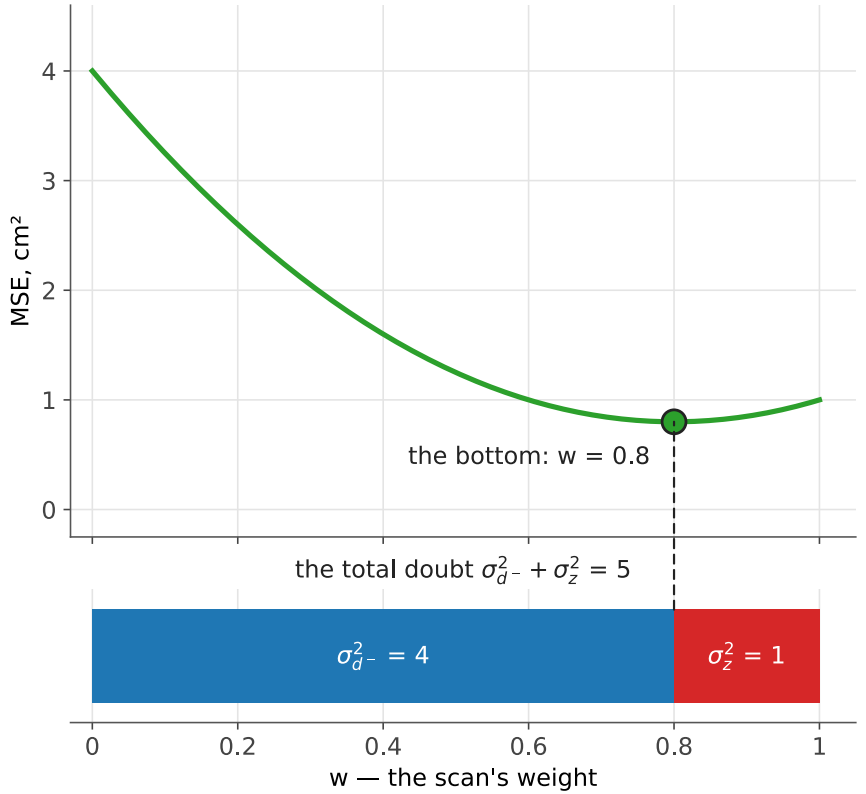
THE BOTTOM OF THE PARABOLA IS THE KALMAN GAIN K

$$K = \frac{\sigma_{d^-}^2}{\sigma_{d^-}^2 + \sigma_z^2}$$

The best weight: the prediction's share of the total doubt.

Three pairs of noises, cm² (example); the picture is the first:

CASE	$\sigma_{d^-}^2$	σ_z^2	K
wheels unsure	4	1	$4/(4+1)$ = **0.8**
lidar unsure	1	4	$1/(1+4)$ = **0.2**
equally sure	1	1	$1/(1+1)$ = **0.5**



LEGEND

green curve — the mix's error, MSE, for every weight w of the scan; **green dot** — its bottom, the best weight

dashed line — from the bottom down to the bar: the bar splits right under it

bar — the total doubt $\sigma_{d^-}^2 + \sigma_z^2$: **blue** — the prediction's part, **red** — the lidar's

K — the Kalman gain: the best weight of the scan, from 0 to 1

$\sigma_{d^-}^2$ — the prediction's variance: its typical error, squared

σ_z^2 — the lidar's variance

MSE — mean squared error, cm²

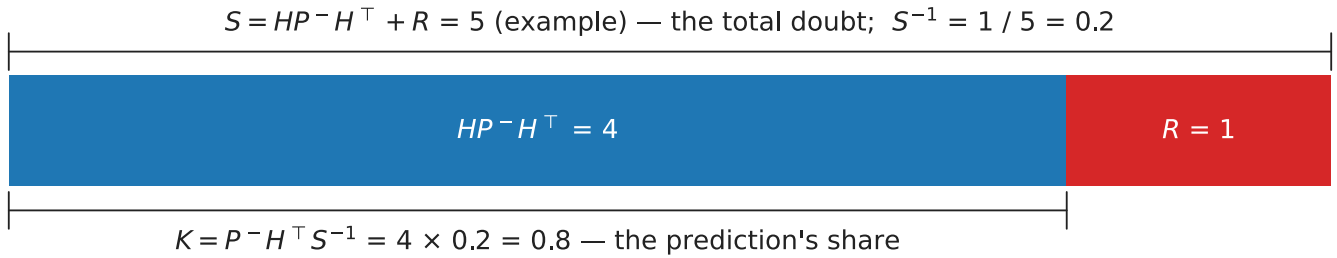
$\sigma_{d^-} = 0 \to K = 0$: keep the prediction · $\sigma_z = 0 \to K = 1$: take the scan

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/04-One-Dimensional-Kalman-Filters.ipynb

THE KALMAN GAIN FOR ANY SENSOR, PIECE BY PIECE

Libraries such as filterpy write the gain for any sensor: $K = P^- H^\top (H P^- H^\top + R)^{-1}$. H says what the sensor sees of the state. Cut into pieces:

1. **The prediction's doubt, as the sensor sees it:** $H\,P^-\,H^\top$ — a variance through a multiplication; for one number $H^2\,P^-$
2. **The whole doubt:** $S = H\,P^-\,H^\top + R$ — the prediction's part plus the sensor's part
3. **The prediction's share of that whole:** $K = P^-\,H^\top\,S^{-1}$
4. **Our lidar sees the distance itself,** $H = 1$: $K = \dfrac{\sigma_{d^-}^2}{\sigma_{d^-}^2 + \sigma_z^2}$ — the bottom of the parabola



github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/06-Multivariate-Kalman-Filters.ipynb

LEGEND

bar — S (example): **blue** — the prediction's part, **red** — the lidar's; bracket over the bar — S, under the blue part — K

K — the Kalman gain, the scan's weight

H — what the sensor sees of the state: here the distance itself, $H = 1$

P^- — the prediction's variance, our $\sigma_{d^-}^2$

R — the sensor's variance, our σ_z^2

$HP^-H^\top$ — the variance rule: $y = Hx$ has $\sigma_y^2 = H^2\sigma_x^2$; $H^\top$ — H turned over

S — the whole doubt; S^{-1} — one over S

FILTERPY COMPUTES K ON OUR DRIVE

Our drive, the second scan: $\sigma^2_{d^-} \approx 0.61$ cm², $\sigma^2_z \approx 1.02$ cm², $H = 1$. What is K?

Python · filterpy/kalman/kalman_filter.py — filterpy's KalmanFilter.update, library code

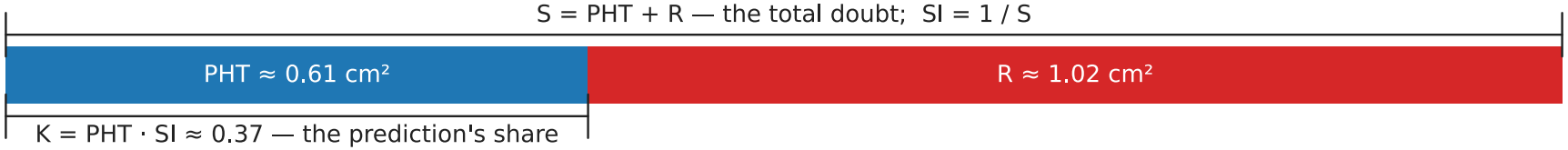
```
# common subexpression for speed
PHT = dot(self.P, H.T)

# S = HPH' + R
# project system uncertainty into measurement space
self.S = dot(H, PHT) + R
self.SI = self.inv(self.S)
# K = PH'inv(S)
# map system uncertainty into kalman gain
self.K = dot(PHT, self.SI)
```

$P^- H^\top \approx 0.61$ cm²

$S = HP^-H^\top + R \approx 0.61 + 1.02$ cm²
$S^{-1} = 1/S$

$K = P^- H^\top S^{-1} \approx 0.61 / (0.61 + 1.02) \approx$ **0.37**



LEGEND

blue — the prediction's part, red — the lidar's: in the code, the formulas and the bar

K — the Kalman gain, the scan's weight: `self.K`

P^- — filterpy's prediction variance, our $\sigma^2_{d^-}$: `self.P` before the update

R — filterpy's lidar variance, our σ^2_z: `R`

H — what the sensor sees of the state: 1, the lidar sees the distance itself

$H^\top$ — H turned over: `H.T`

`PHT` — $P^- H^\top$, computed once for speed

S — the whole doubt: `self.S`

S^{-1} — one over S: `SI`, `inv`

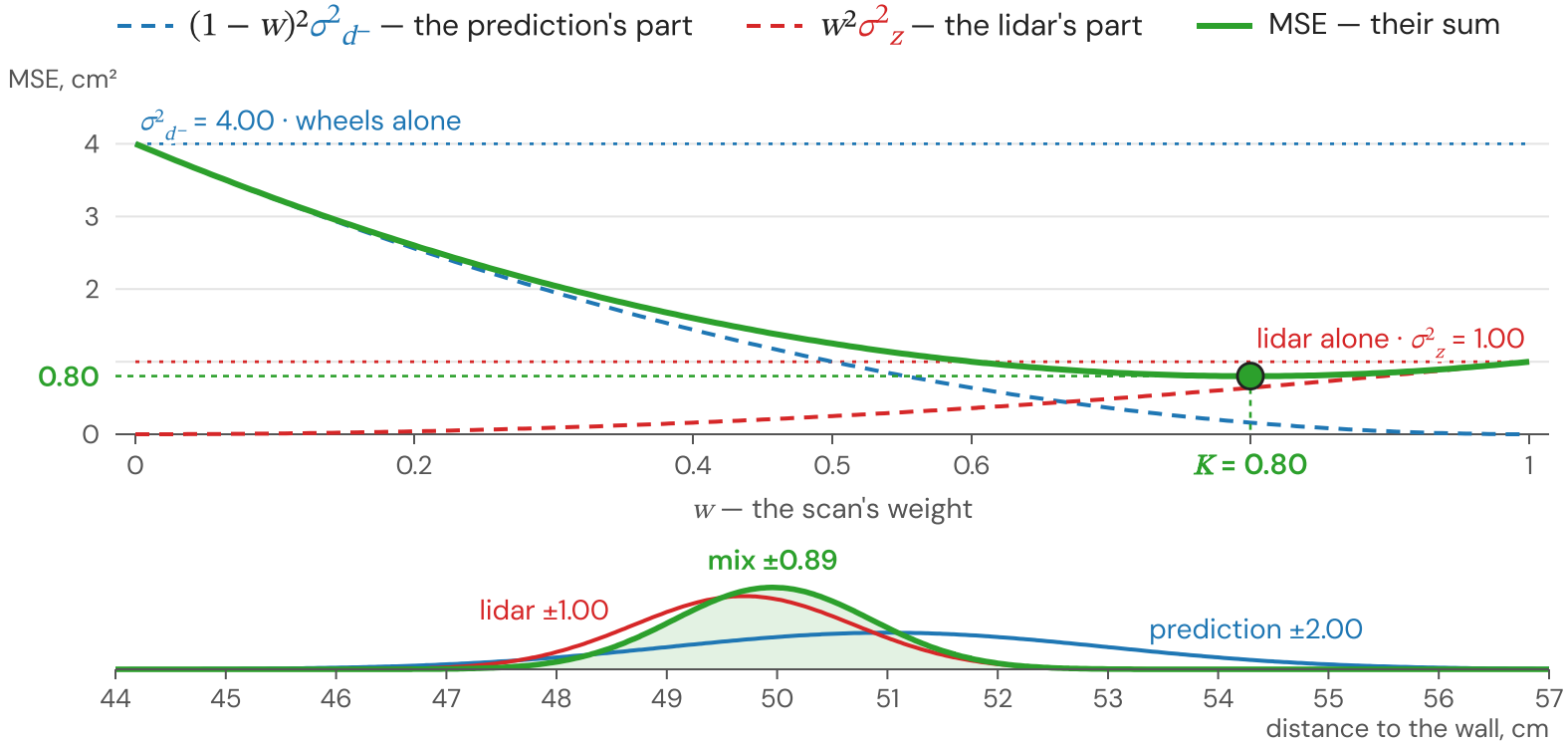
`dot` — the product of two arrays

cm² — the unit of the variances and of S; K has none

🎯 github.com/rlabbe/filterpy

MOVE THE NOISES: K FOLLOWS THE SURER SENSOR

σ_z — the lidar's noise: **we set it once**, measured standing · σ_{d^-} — the prediction's doubt: **the filter computes it every scan**, the last estimate's doubt plus one step's — so K is new every scan. Start (example): σ_{d^-} = 2 cm, σ_z = 1 cm.



the wheels' prediction — the filter computes it

σ_{d^-} 2.00 cm

$\sigma^2_{d^-} = 2.00^2 = 4.00$ cm²

the lidar — we set it once

σ_z 1.00 cm

$\sigma^2_z = 1.00^2 = 1.00$ cm²

the best weight: $K = \sigma^2_{d^-}/(\sigma^2_{d^-} + \sigma^2_z)$
$= 4.00/(4.00 + 1.00) = 0.80$

$\sigma^2_{\hat{d}}$ = MSE at the bottom = $(1-K)\sigma^2_{d^-}$
$= (1 - 0.80) \cdot 4.00 = 0.80$ cm²

below both $\sigma^2_{d^-}$ and σ^2_z: the mix is surer than either sensor

reset

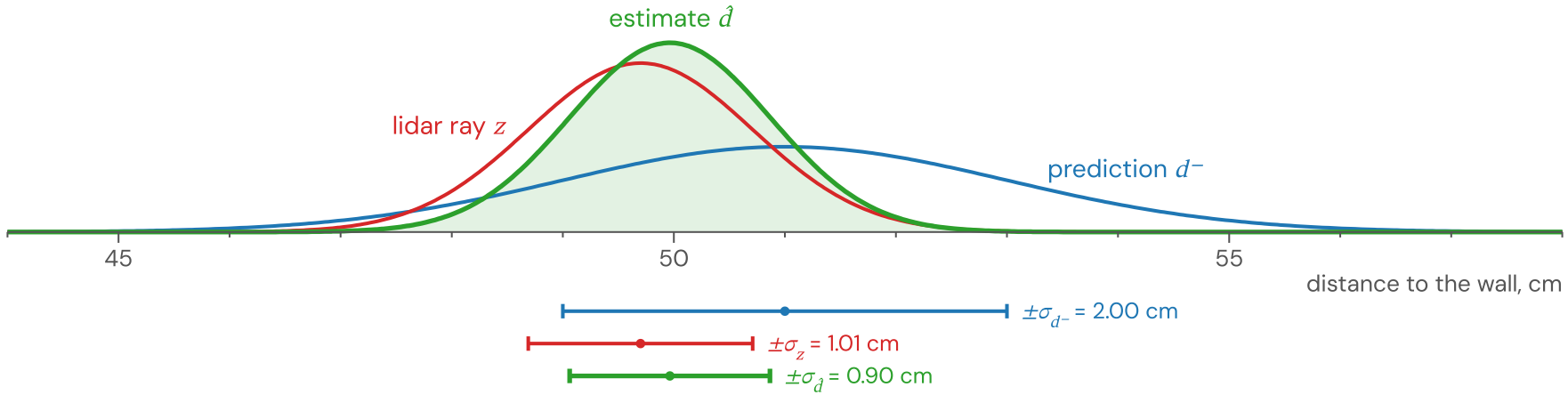
TWO GAUSSIANS GIVE A NARROWER GAUSSIAN

$$\hat{d} = d^- + K\,(z - d^-)$$
$$\sigma^2_{\hat{d}} = (1 - K)\,\sigma^2_{d^-}$$

Python · filterpy/stats/stats.py · mul — filterpy's helper: the product of two Gaussians, library code

```
mean = (var1*mean2 + var2*mean1) / (var1 + var2)
var = 1 / (1/var1 + 1/var2)
```

As on the grid, the update multiplies the belief by the scan's Gaussian.



prediction

d^- 51.0 cm

σ_{d^-} 2.0 cm

lidar ray

z 49.7 cm

σ_z 1.0 cm

reset

estimate: $\hat{d}$ = 49.96 cm, $\sigma_{\hat{d}}$ = 0.90 cm

LEGEND

blue — the prediction, red — the forward ray, green — the estimate

d^-, σ_{d^-} — the prediction, its typical error

z, σ_z — the ray's scan, its typical error

$\hat{d}$, $\sigma_{\hat{d}}$ — the estimate after the update, its typical error

σ^2 — a variance; ±σ holds 68 %

K — the Kalman gain, the scan's best weight

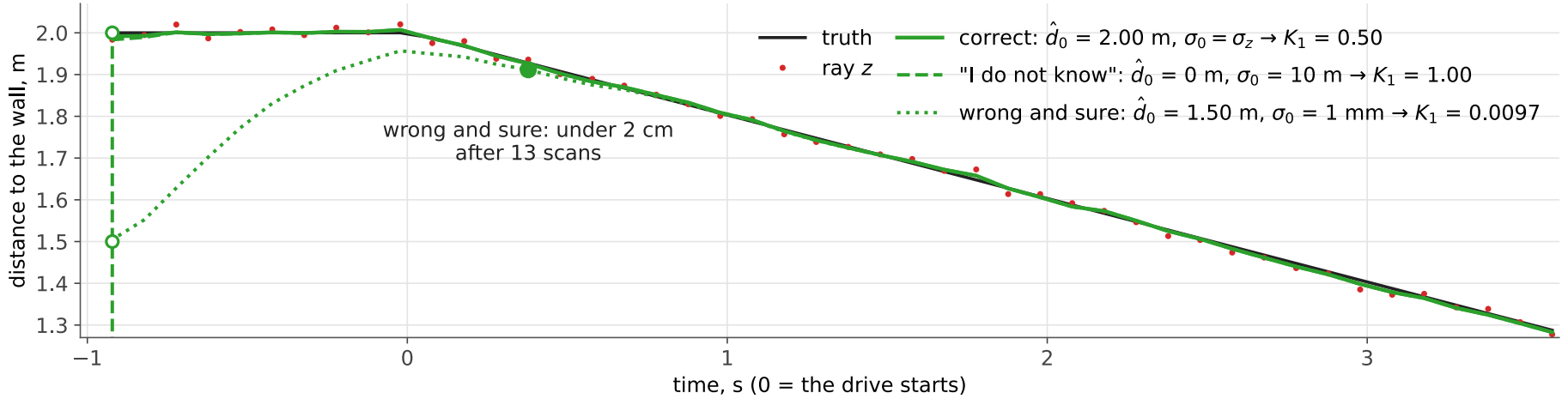
start: 51 ± 2 cm (example); the ray 49.7 ± 1.01 cm, measured

we set σ_z once; d^-, σ_{d^-}, K — the filter computes every scan

`mean1`, `var1` — d^-, $\sigma^2_{d^-}$; `mean2`, `var2` — z, σ^2_z; `mean`, `var` — $\hat{d}$, $\sigma^2_{\hat{d}}$

www.bzarg.com/p/how-a-kalman-filter-works-in-pictures/

STARTING THE FILTER: THE FIRST GUESS AND ITS DOUBT



$$K_1 = \frac{\sigma_0^2}{\sigma_0^2 + \sigma_z^2}$$

The first scan's weight: the start's doubt stands where the prediction's was.

- Before the first scan **we set** the start: a first guess $\hat{d}_0$ and its doubt σ_0
- **"I do not know"**, a large σ_0: $K_1 \approx 1$, the first scan decides at once
- **Wrong and sure**, 0.5 m short with σ_0 = 1 mm: $K_1 \approx 0.01$, the scan barely counts; 13 scans to come within 2 cm
- Not sure where the robot is? A large σ_0 — in `robot_localization`, `initial_estimate_covariance`

LEGEND

green lines — the filter's estimate from three starts: solid — correct, dashed — "I do not know", dotted — wrong and sure; hollow dots — the starts; the bold dot — within 2 cm

black — the truth; red dots — the scans z of the forward ray

$\hat{d}_0$, σ_0 — the start: the first guess and its typical error

K_1 — the Kalman gain of the first scan

σ_z — the lidar's typical error, 10.1 mm on our drive

small σ_0 → small K_1: the scan barely counts

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/04-One-Dimensional-Kalman-Filters.ipynb

FORMULA ↔ FILTERPY: THE LOOP

predict $d^- = \hat{d} - v\,\Delta t \qquad \sigma_{d^-}^2 = \sigma_{\hat{d}}^2 + \sigma_u^2$

update $\hat{d} = d^- + K\,(z - d^-)$

$\sigma_{\hat{d}}^2 = (1 - K)\,\sigma_{d^-}^2$

- blue — the prediction, red — the scan, green — the estimate: in the formulas and in the code
- $\hat{d}, \sigma_{\hat{d}}$ — the estimate, its typical error; d^-, σ_{d^-} — the prediction, its error; σ_u — one step's error; z, σ_z — the scan, its error; K — the Kalman gain
- filterpy's names: `x` — the state, our d^- after predict and $\hat{d}$ after update; `P` — its variance, $\sigma_{d^-}^2$ and $\sigma_{\hat{d}}^2$; `Q` — σ_u^2; `R` — σ_z^2; `y` — the innovation $z - d^-$
- `dot` — a product of arrays (× for numbers); `.T` — $^\top$; `self._I` — the identity matrix I (1 for a number)
- **we set** `Q` and `R` once; `x`, `P` and `K` **the filter computes** every step

Python · filterpy/kalman/kalman_filter.py — filterpy's KalmanFilter.predict() and update(), library code

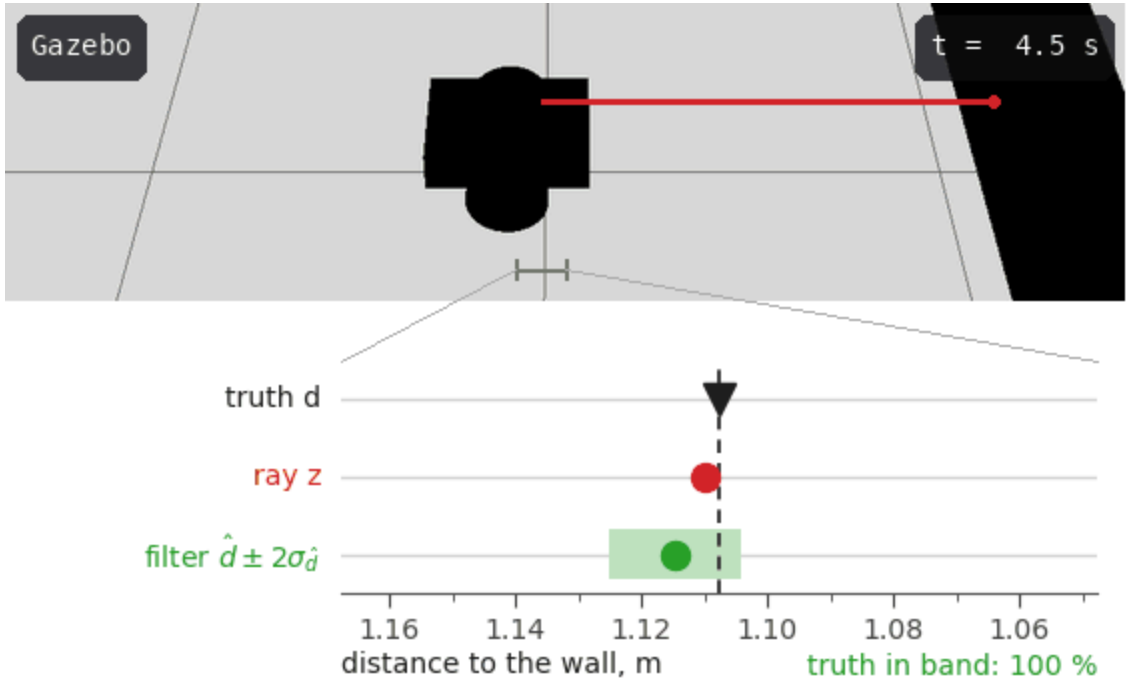
```
# predict
# x = Fx + Bu
self.x = dot(F, self.x) + dot(B, u)
# P = FPF' + Q
self.P = self._alpha_sq * dot(dot(F, self.P), F.T) + Q

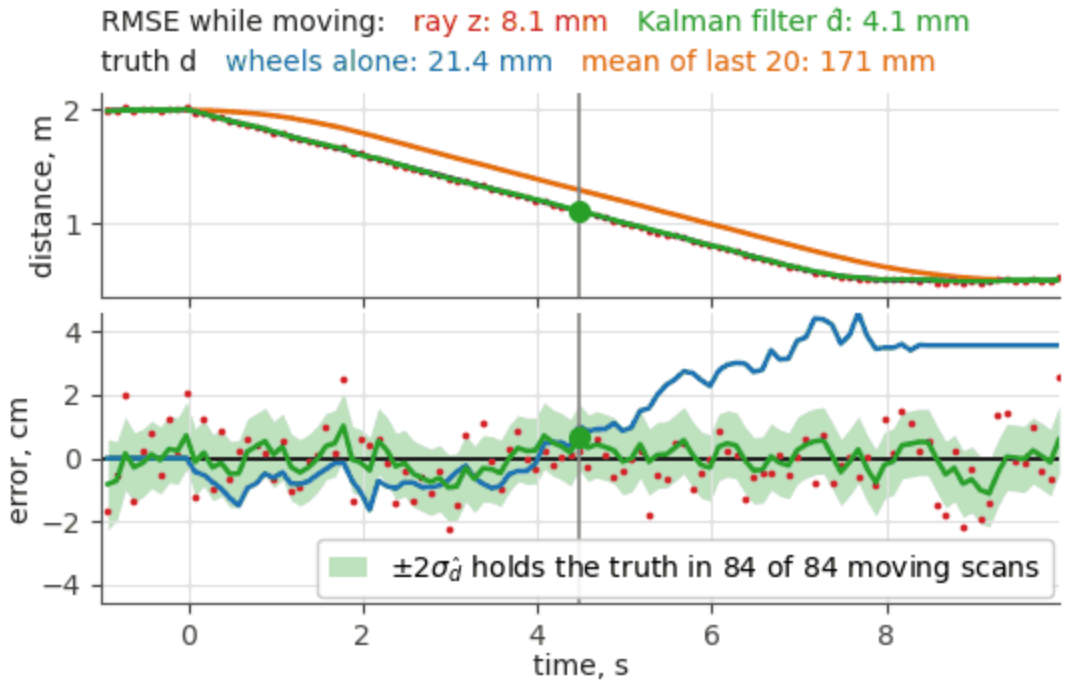
# update: S, SI and K as computed on our drive
self.y = z - dot(H, self.x)
self.x = self.x + dot(self.K, self.y)
I_KH = self._I - dot(self.K, H)
self.P = dot(dot(I_KH, self.P), I_KH.T) + dot(dot(self.K, R), self.K.T)
```

- $B\,u$ — the motion command: $B = 1, u = -v\,\Delta t$, the step toward the wall
- F — how the state moves by itself: here 1; $FPF^\top$ — the same variance rule as $HP^-H^\top$, for one number F^2P: here just `P`
- H — what the sensor sees: 1; `_alpha_sq` — fading memory, 1 by default
- `I_KH` is $1 - K$; the last line is $(1 - K)\,\sigma_{d^-}^2$ written longer, so it stays right for any K

🎯 github.com/rlabbe/filterpy

THE FILTER ON OUR DRIVE: NO LAG, LESS NOISE

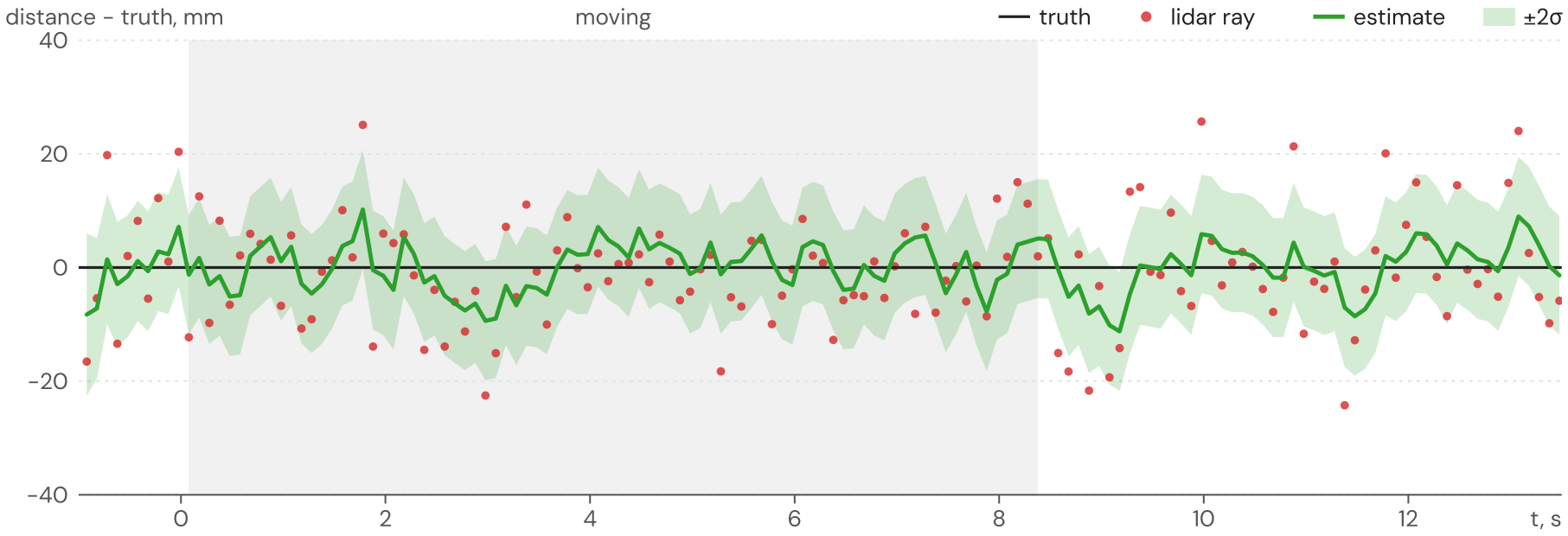




- RMSE (root mean square error) while driving: filter **4.1 mm** · ray 8.1 · wheels 21.4 · moving average 171 mm
- No lag: the filter predicts the motion instead of waiting for it; its band held the truth in all 84 scans
- Grey bracket under the robot — the 12 cm the lanes zoom into. Black — the truth d; red — the ray z; blue — the wheels; orange — the mean of the last 20 scans; green — the filter's $\hat{d}$ and its band $\pm 2\sigma_{\hat{d}}$; grey line — the frame's moment

TUNING: THE LIDAR'S NOISE AND THE WHEELS' NOISE

$\sigma_z = 10.1$ mm, measured standing $\sigma_u = \sigma_v \, \Delta t = 0.032 \text{ m/s} \times 0.1 \text{ s} = 3.2$ mm



both noises as measured · K at the end 0.268 · while moving: RMSE 4.1 mm, truth inside ±2σ 100 % (84 of 84)

σ_u^2/σ_z^2 × 1 (measured) reset

× 0.01 × 0.1 × 1 (measured) × 10 × 100

youtu.be/ul3u2yLPwU0

LEGEND

black — the truth, zero error; red dots — each scan's error; green — the estimate's error, its band $\pm 2\sigma_{\hat{d}}$

σ_z — the lidar's typical error, measured standing

σ_u — the error one step adds: σ_v, the wheels' speed error, × Δt, the time between scans; checked on this drive against Gazebo's truth: 3.1 mm per step

we set both once; filterpy and robot_localization call σ_z^2 `R`, σ_u^2 `Q`

σ_u too big — the estimate shakes with the ray; too small — it follows the wheels

only the ratio σ_u^2/σ_z^2 — the slider — moves the estimate; their size sets how sure the filter is

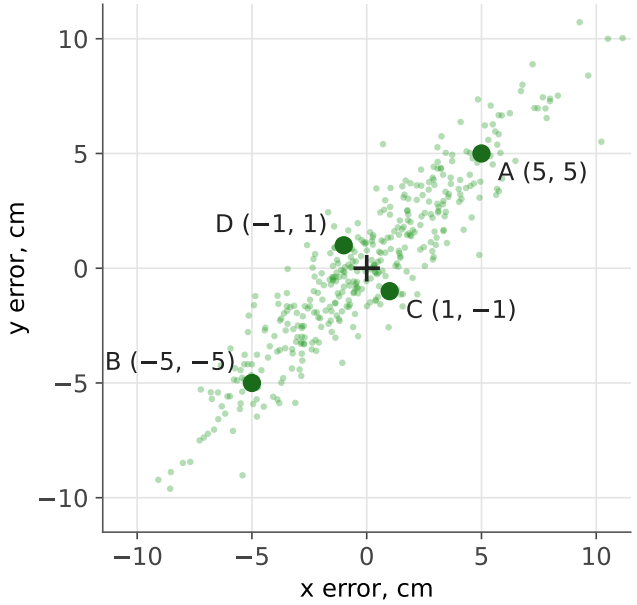
K — the scan's weight; $\sigma_{\hat{d}}$ — the estimate's typical error

FROM A DISTANCE TO A POSE: THE COVARIANCE MATRIX

Four position errors of the robot, cm (example):

POINT	x	y	$x \cdot y$
A	5	5	25
B	-5	-5	25
C	1	-1	-1
D	-1	1	-1
mean			**12**

$$\sigma_x^2 = \sigma_y^2 = (25 + 25 + 1 + 1)/4 = 13$$
$$\rho = 12/13 \approx 0.92$$



$$P = \begin{bmatrix} \sigma_x^2 & \mathrm{cov}(x,y) \\ \mathrm{cov}(x,y) & \sigma_y^2 \end{bmatrix} = \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix}$$

LEGEND

light green dots — many position errors; dark dots A–D — the four in the table

the pose — (x, y, θ), θ — the yaw; here only x, y: the position errors, cm

P — the covariance matrix: the doubt of several numbers, 3 × 3 for (x, y, θ)

σ_x^2, σ_y^2 — the variance of each, cm²: on the diagonal

$\mathrm{cov}(x, y)$ — the mean of $x \cdot y$: do the errors go together? Off the diagonal

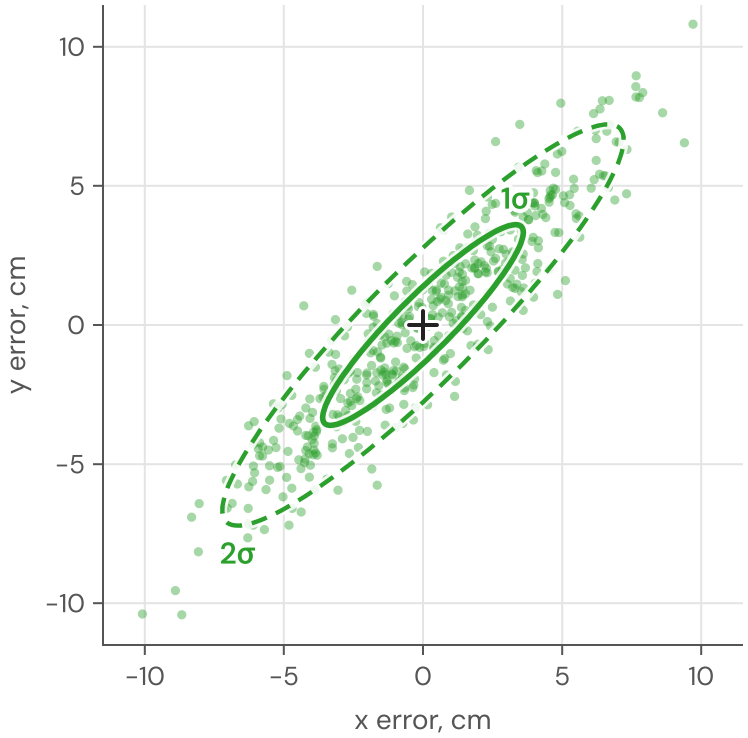
$\rho = \frac{\mathrm{cov}(x,y)}{\sigma_x \sigma_y}$ — correlation, from -1 to 1

black cross — zero error: the true position

The filter must know which errors go together — a heading error becomes a sideways error.

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/05-Multivariate-Gaussians.ipynb

THE COVARIANCE ELLIPSE: $\pm\sigma$ IN 2D



σ_x 3.61 cm

σ_y 3.61 cm

ρ 0.92

$$P = \begin{bmatrix} \sigma_x^2 & \rho\sigma_x\sigma_y \\ \rho\sigma_x\sigma_y & \sigma_y^2 \end{bmatrix} = \begin{bmatrix} 13.0 & 12.0 \\ 12.0 & 13.0 \end{bmatrix} \text{ cm}^2$$

— inside 1σ: **197** of 500 points — **39 %**

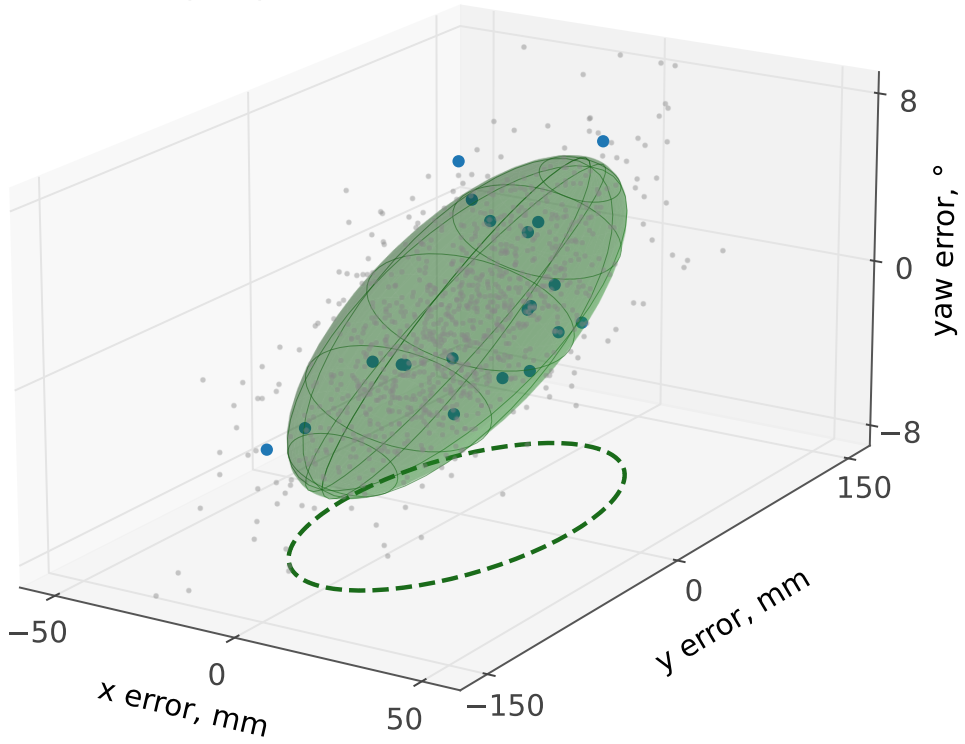
- - - inside 2σ: **432** of 500 points — **86 %**

reset

- **The 2D case:** x, y only (with the yaw θ, an ellipsoid — next); 1σ holds **39 %** of the points, 2σ — **86 %**
- ρ — the correlation, -1 to 1: $\rho = 0$ — along the axes; $\rho \to \pm 1$ — a slanted line; $\rho\sigma_x\sigma_y$ — the same $\mathrm{cov}(x, y)$
- σ_x, σ_y — the typical error along x, y; P — their covariance matrix, cm²; start: the previous slide's P (example)

THREE VARIABLES: THE ELLIPSE BECOMES AN ELLIPSOID

- 1000 simulated drives
- 20 Gazebo runs
- the 2σ ellipsoid
- its shadow: the x–y ellipse



- The ellipse was the **2D** case: x and y only
- Our state has **three** numbers, x, y, θ : the covariance P is 3 × 3, and its ±σ surface is an **ellipsoid**
- It leans: a heading error goes with a sideways error — y and θ are tied
- Its shadow on the floor is the x–y ellipse — what RViz will draw later today

LEGEND

grey dots — the ends of 1000 simulated drives of 2 m, each with our wheels' noise

blue dots — where the wheels say each of 20 Gazebo runs ended: one true drive, its own noise on the wheels' readings

green surface — their 2σ ellipsoid: two typical errors in every direction

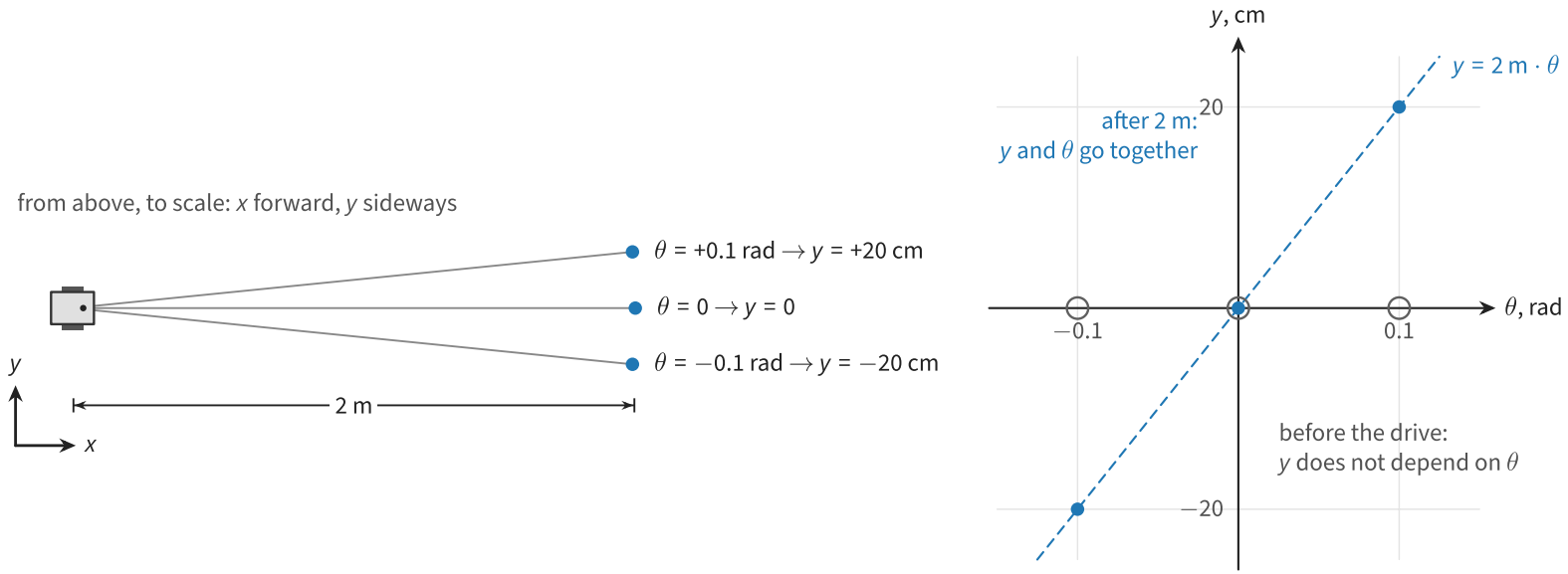
on the floor — its shadow, the 2σ ellipse of x and y alone

x, y — the position errors, mm; θ — the yaw error, °

P — the covariance matrix of (x, y, θ), 3 × 3

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/05-Multivariate-Gaussians.ipynb

A HEADING ERROR BECOMES A SIDEWAYS ERROR



$$F = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \qquad P = \begin{bmatrix} 0 & 0 \\ 0 & 0.01 \end{bmatrix} \qquad P^- = F\,P\,F^\top = \begin{bmatrix} 0.04 & 0.02 \\ 0.02 & 0.01 \end{bmatrix}$$

One step carries the doubt: the heading's doubt becomes a sideways doubt and ties y to θ.

- F — one step of 2 m: y gains 2 × the heading error, θ stays
- Top left, $2^2 \times 0.01$ = 0.04 m²: sideways ±20 cm · off the diagonal, 2×0.01 = 0.02: y and θ now go together

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/06-Multivariate-Kalman-Filters.ipynb

LEGEND

grey lines — copies of the robot, 2 m each, each with its own heading error; blue dots — where they end; grey rings — the same errors before the drive

x — forward, y — sideways; θ — the yaw; the robot was meant to face $\theta = 0$, so here θ is its heading error, rad: ±0.1 rad ≈ ±6° (example)

P, P^- — the covariance of (y, θ) before and after the step; y in m, θ in rad

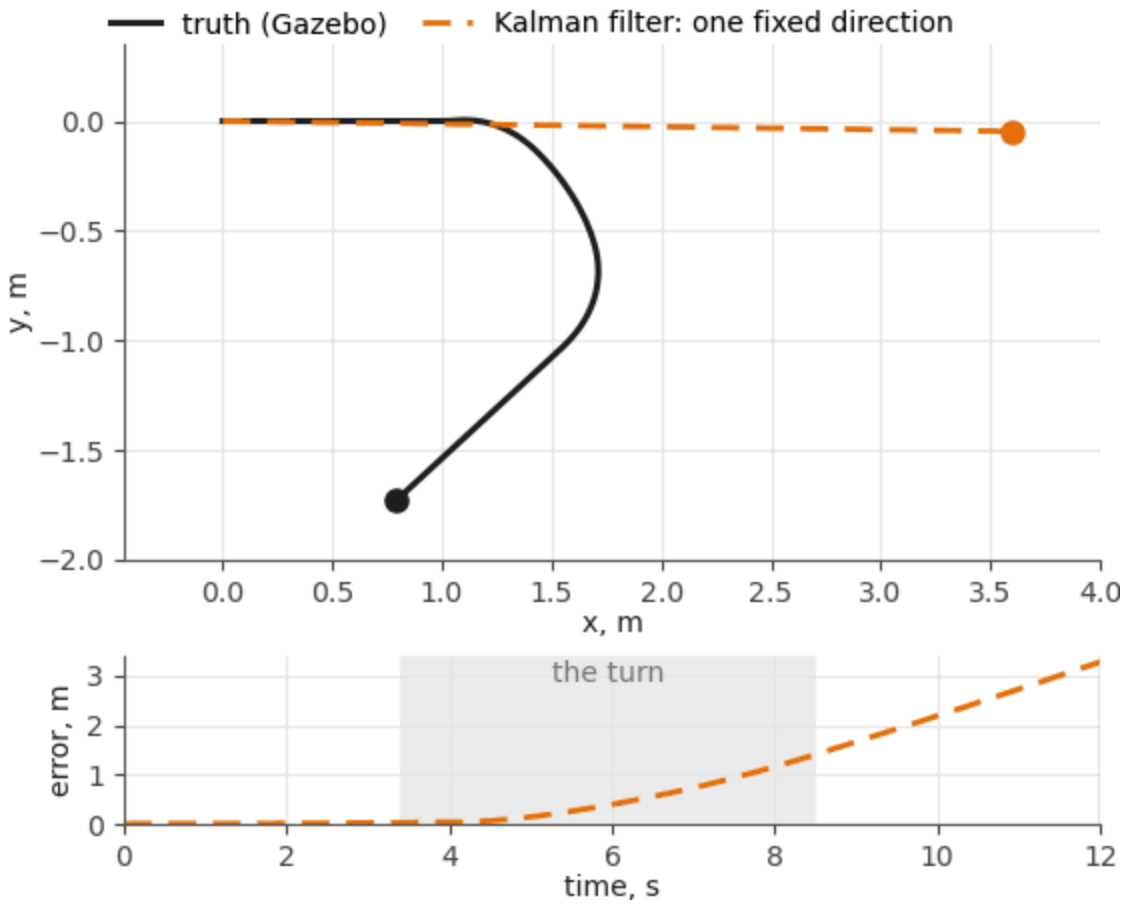
F — the step's table of slopes: how a small error of each variable moves each variable

$F^\top$ — F turned over: rows become columns

$FPF^\top$ — the variance rule for several numbers, as $HP^-H^\top$: for one number $a\,\sigma^2 a = a^2\sigma^2$

small angles: sideways = 2 m × θ, as $\sin\theta \approx \theta$

THE KALMAN FILTER LOSES A TURNING ROBOT



- Our Kalman filter's motion was a straight line: at the wall, d moved by $-v\,\Delta t$. On the floor that is **one fixed direction**
- **It knows** the wheels' speed v and even the yaw θ (the gyroscope); **nothing measures** x, y — only speeds, as later in robot_localization
- On the straight part it follows the truth
- In the turn it goes on straight: its error grows by metres
- One step of a turning robot: $v\,\Delta t\cos\theta$ along x, $v\,\Delta t\sin\theta$ along y — curves in θ

LEGEND

black — the truth, Gazebo's ground truth; orange dashed — the Kalman filter with one fixed direction (`filterpy`); below — its position error over time, the turn shaded

dots — where each is now; grey vertical line on the error plot — now

x, y — the position on the floor, m

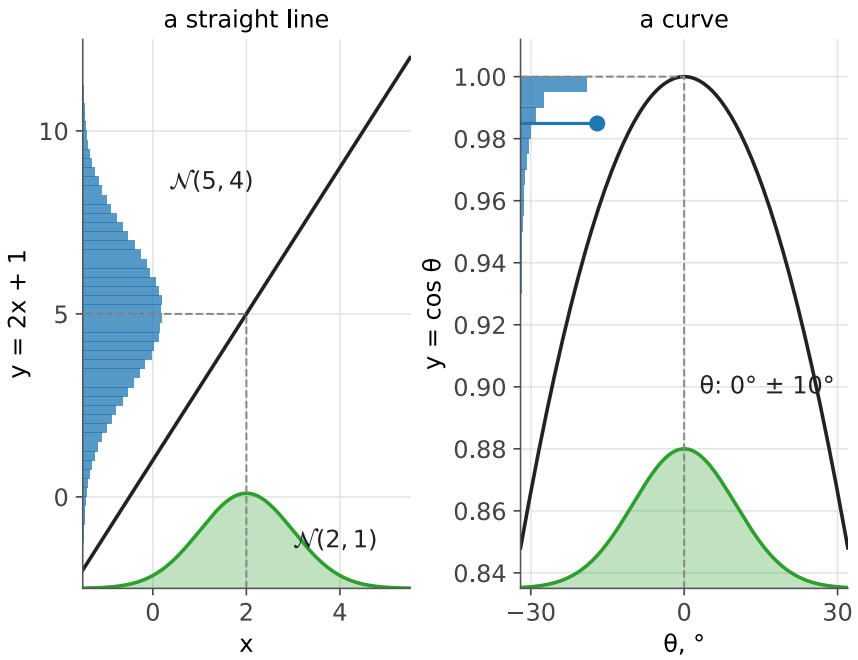
θ — the yaw, the direction the robot faces

$v\,\Delta t$ — one step: speed × time

d — the distance to the wall, our 1D state

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/11-Extended-Kalman-Filters.ipynb

A GAUSSIAN THROUGH A LINE AND THROUGH A CURVE



	A STRAIGHT LINE $y = a\,x + c$	A CURVE $y = \cos\theta$
what comes out	a Gaussian again	not a Gaussian: bent, pressed against 1
its spread	$\sigma_y^2 = a^2\sigma_x^2$, exactly	depends on where on the curve
its centre	the line at the old centre	below the curve at the old centre
example	$\mathcal{N}(2,1) \to \mathcal{N}(5,4)$	yaw 0 ± 10° → a hump below 1
the Kalman filter	exact: $\sigma_{d^-}^2 = a^2\sigma_{\hat{d}}^2 + \sigma_u^2$	needs help — next slide

- A turning robot steps by $v\,\Delta t\cos\theta$ and $v\,\Delta t\sin\theta$ — curves in its yaw; the bigger the doubt in θ, the more they bend the Gaussian

LEGEND

green — the Gaussian that goes in; blue histogram — what comes out: 200 000 random inputs pushed through; black — the line and the curve; blue dot — the mean of what comes out, 0.985

grey dashed — the old centre carried through: 2 → 5, 0° → 1

x, y — what goes in and what comes out

a — the line's slope; c — its shift

σ_x, σ_y — the typical error before and after; σ^2 — a variance

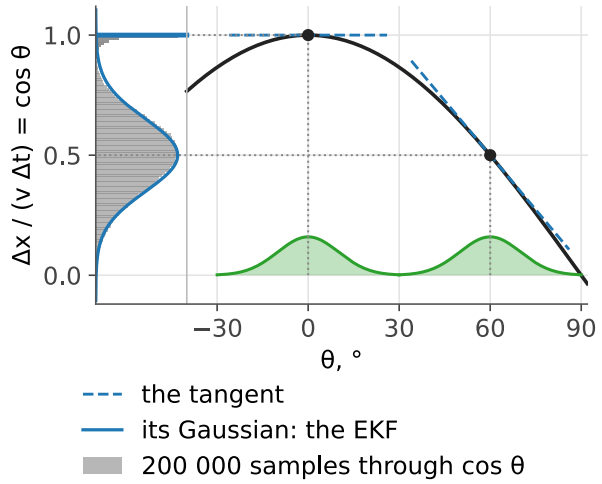
$\mathcal{N}(2,1)$ — a normal distribution: centre 2, variance 1

d^-, $\hat{d}$ — the predicted and the estimated distance; σ_{d^-}, $\sigma_{\hat{d}}$ — their errors; σ_u — one step's error; at the wall $a = 1$

θ — the yaw; $v\,\Delta t$ — one step stretch by a: σ grows a times, σ² — a^2 times

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/09-Nonlinear-Filtering.ipynb

THE EKF REPLACES THE CURVE WITH ITS TANGENT



$$\Delta x = v\,\Delta t\cos\theta \quad \rightarrow \quad \frac{\partial\,\Delta x}{\partial\theta} = -v\,\Delta t\sin\theta$$

The tangent is straight, so the Kalman filter works again.

At 60° the tangent works; at 0° it misses the bend.

A flat robot: `cp` = 1, `sy` = $\sin\theta$, so the slope is $-\sin\theta \cdot v\,\Delta t$

C++ · robot_localization/src/ekf.cpp — the predict step of the EKF inside ekf_node

```
// the motion: x grows by cos(yaw) · vx · Δt; (StateMemberX, StateMemberVx) — the table's cell: row x, column vx
transfer_function_(StateMemberX, StateMemberVx) = cy * cp * delta_sec;
// its slope in yaw, ∂/∂yaw: the tangent, -sin(yaw)
x_coeff = -sy * cp;
double dFx_dY =
  (x_coeff * x_vel + y_coeff * y_vel + z_coeff * z_vel) * delta_sec + ...
...
transfer_function_jacobian_(StateMemberX, StateMemberYaw) = dFx_dY;  // into F: row x, column yaw
```

🎯 github.com/cra-ros-pkg/robot_localization

LEGEND

black — $\cos\theta$; **blue dashed** — tangents at 60° and 0°; **green** — the yaw's Gaussian, ±10°; black dots — the estimate, dotted — carried left

blue curve, left — the EKF's Gaussian, through the tangent; grey — 200 000 yaws through $\cos\theta$: the truth

Δx — the step along x; $v\,\Delta t$ — speed × time; θ — the yaw

tangent — touches the curve at the estimate; ∂ — its slope when only θ changes: $(\cos\theta)' = -\sin\theta$

`transfer_function_` — the motion: it moves the estimate

`sy`, `cy`, `cp` — sin, cos of yaw, cos of pitch; `x_vel`, `delta_sec` — v, Δt; `y_vel`, `z_vel` — 0

`x_coeff` — the slope of cos;

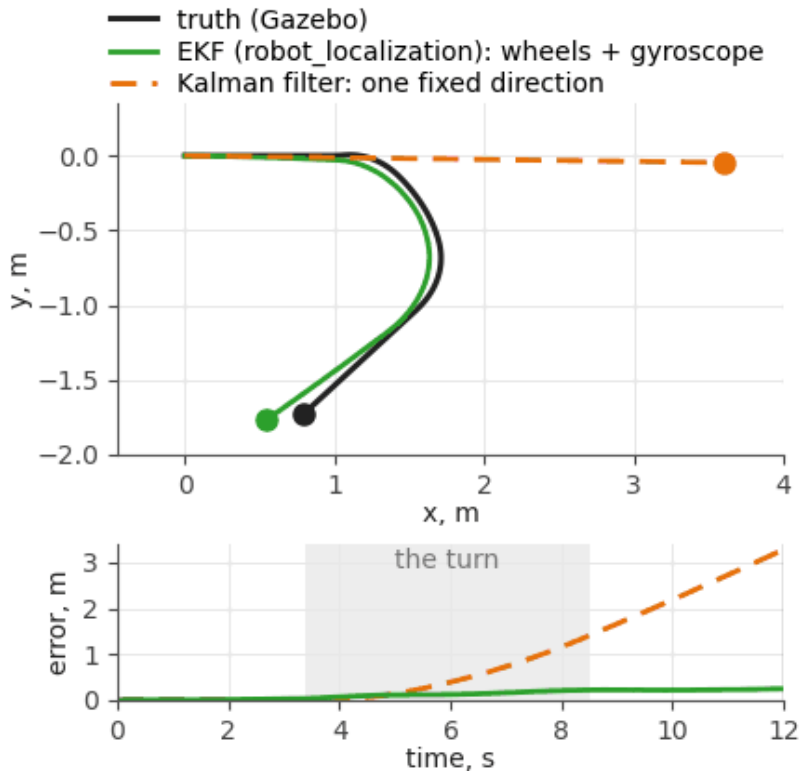
`dFx_dY` — the slope of x in yaw, a cell of `transfer_function_jacobian_`, our F

KALMAN FILTER VS EKF: ONLY THE PREDICT CHANGES

	KALMAN FILTER	EKF
the motion	a line: $d^- = \hat{d} - v\,\Delta t$	a curve: $x^- = \hat{x} + v\,\Delta t \cos\hat{\theta}$
F in predict	fixed: 1 for the wall	the tangent at the estimate, new every step
K and the update	the same	the same
what it tracks	the distance to the wall	a robot that turns: (x, y, θ)
ready	`filterpy` `KalmanFilter`	`robot_localization` `ekf_node`

$$P^- = F\,P\,F^\top + Q$$

The EKF takes a new F, the tangent at the estimate, every step.



LEGEND

black — the truth; orange dashed — the Kalman filter; green — the EKF, wheels + gyroscope; dots — now; below — errors, grey band — the turn

P, P^- — the covariance of the estimate and of the prediction: σ^2 of x, y, θ on the diagonal

F — the step's table of slopes: 1 at the wall; when turning, like $-v\,\Delta t \sin\theta$

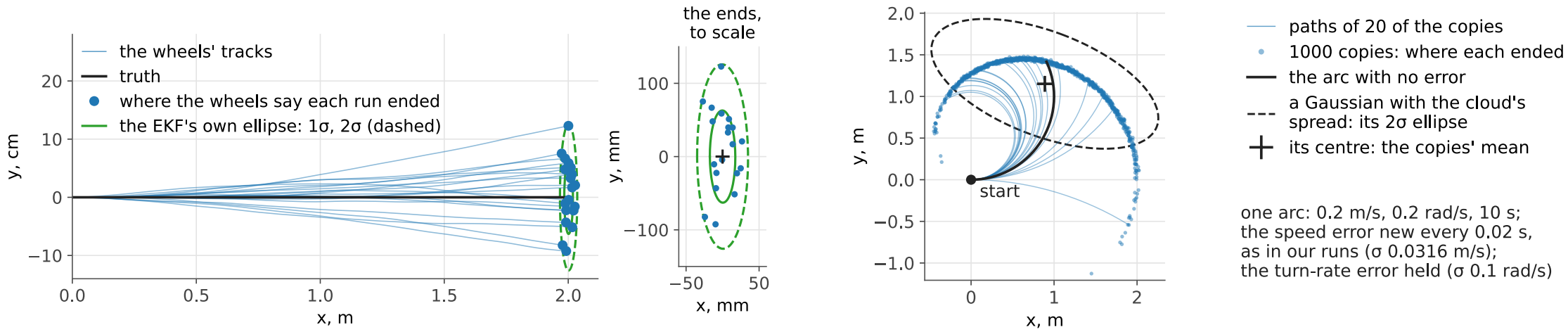
$F^\top$ — F turned over; $FPF^\top$ — the variance rule, $a^2\sigma^2$ for one number

Q — what one step adds: the wall's σ_u^2, now a matrix, set by us: Δt × `process_noise_covariance`

x^- — the predicted x; $\hat{x}$, $\hat{\theta}$ — the estimates; d^-, $\hat{d}$ — the predicted, the estimated distance; $v\,\Delta t$ — one step; K — the Kalman gain

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/11-Extended-Kalman-Filters.ipynb

MONTE CARLO: MANY RANDOM RUNS CHECK THE EKF'S ELLIPSE

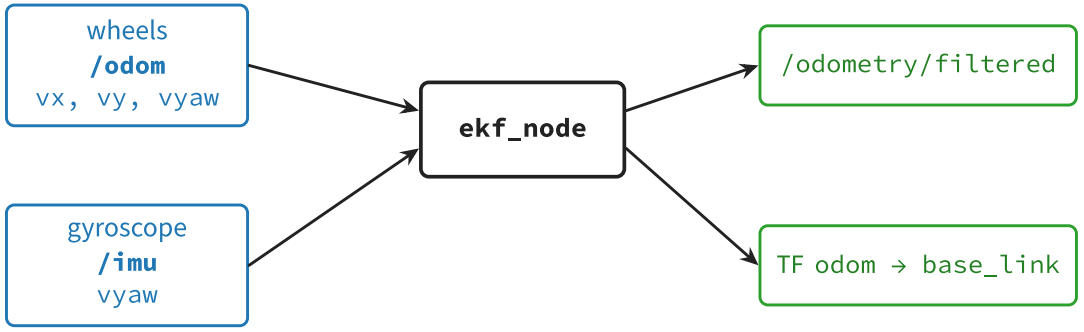


Monte Carlo — a numerical method from probability: instead of a formula, run the motion many times with random errors; the cloud of where the copies end **is** the true belief.

- **Left, the same 2 m drive 20 times:** one truth, only the noise on the wheels' readings differs; the ends spread ±16 mm along, ±55 mm across; the EKF on the wheels alone says ±17 and ±63 mm — 19 of the 20 ends lie inside its 2σ ellipse (right panel: the ends to scale, mm)
- **Right, a 2 m arc:** 1000 copies in Python, the speed error new every 0.02 s, the turn-rate error held — the ends lie on a thin arc, no longer Gaussian; the ellipse with the cloud's spread covers the empty floor inside it and misses the tips. Later: particles in SLAM

github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/09-Nonlinear-Filtering.ipynb

ROBOT_LOCALIZATION: WHAT GOES IN AND OUT



blue — what goes in · green — what comes out

- **Fuse speeds, not poses:** the wheels' pose drifts; their speed is fresh every time
- **The state holds 15 numbers:** pose, speeds, accelerations. The wheels are a measurement here: they update the speed, with their own variance σ^2, from the covariance in `/odom`
- `vx`, `vy`, `vyaw` — the speed forward, sideways and of turning; `vy` = 0: a wheeled robot does not drive sideways — the zero is a reading too

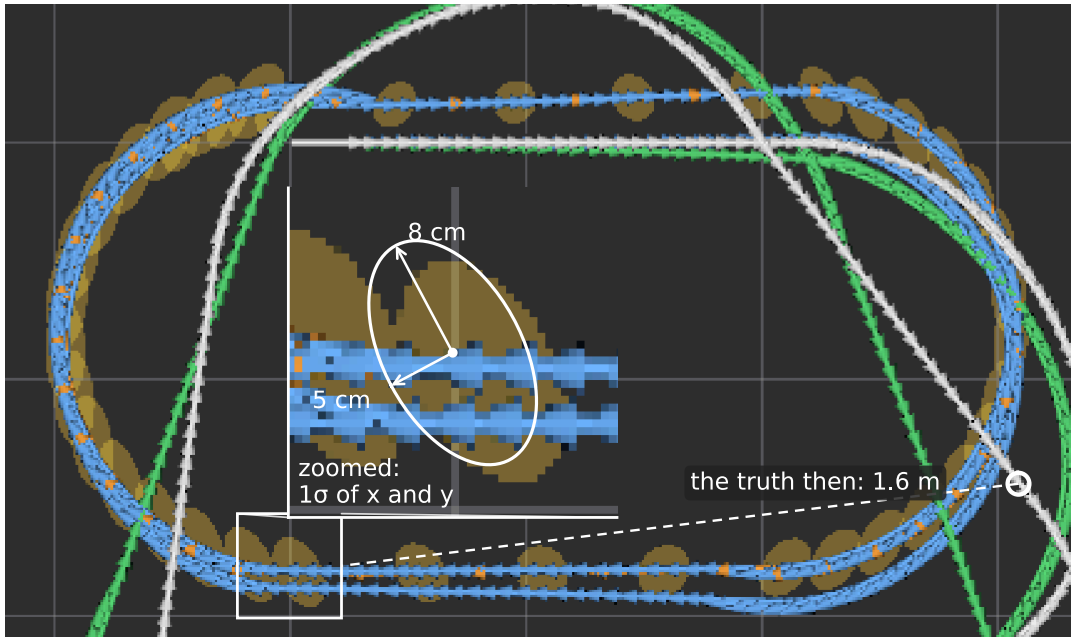
YAML · ekf.yaml — the parameters of robot_localization's ekf_node (excerpt)

```yaml
odom0: /odom
# true — fuse it from /odom, false — ignore
odom0_config: [false, false, false,   # x, y, z
               false, false, false,   # roll, pitch, yaw
               true,  true,  false,   # vx, vy, vz
               false, false, true,    # vroll, vpitch, vyaw
               false, false, false]   # ax, ay, az
two_d_mode: true    # a ground robot: no z, roll, pitch
publish_tf: true    # odom → base_link now comes from here
```

- 15 flags in the state's order: `true` — take this variable from this topic, `false` — leave it out
- `imu0_config` — the same 15 flags, only `vyaw` is `true`
- **Trap:** zeros meant "unknown", but `robot_localization` reads a zero covariance as "perfectly sure" — fill it as a real driver does: your measured σ^2 on the diagonal

docs.nav2.org/jazzy/configuration_and_development/first_time_robot_setup_guide/odom/setup_robot_localization/
github.com/cra-ros-pkg/robot_localization

ROBOT_LOCALIZATION DRAWS THE ELLIPSE

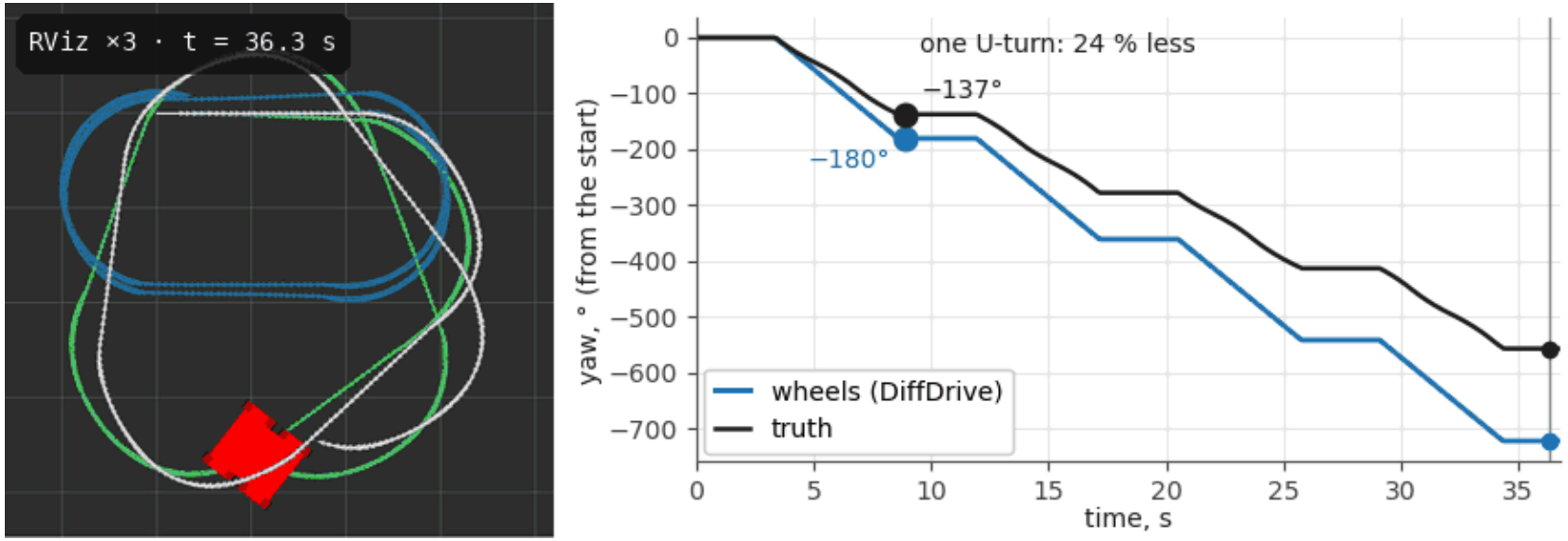


white — the truth · blue — the wheels · yellow ellipses — the EKF on the wheels alone (its orange track lies under the blue) · green — the EKF on the wheels and the gyroscope · white frame — the zoomed ellipse: white outline — its 1σ, the dot — the estimate, arrows — the half-axes · white ring — the truth then

- `ekf_node` publishes its doubt, the covariance P, in `/odometry/filtered`; RViz's stock Odometry display draws it as an ellipse at every pose it keeps
- **One ellipse** (zoomed): the filter's doubt about x, y at that moment, 1σ — half-axes of 8 and 5 cm; by the filter's own account, the robot is inside about 39 % of the time
- Nobody measures x and y, only speeds: the ellipses grow — 1σ about 1 cm after 2 s, 8 × 5 cm after two loops (the position is not observable)
- They stay long roughly across the long straights: the heading doubt times the distance driven becomes a sideways doubt — the EKF's tangent at work; the doubt remembers the whole path, it does not turn with the robot
- The zoomed ellipse says ±8 cm; the truth was 1.6 m away — the filter does not know the wheels slip

docs.nav2.org/jazzy/configuration_and_development/first_time_robot_setup_guide/odom/setup_robot_localization/

FOUR WHEELS SLIP IN TURNS

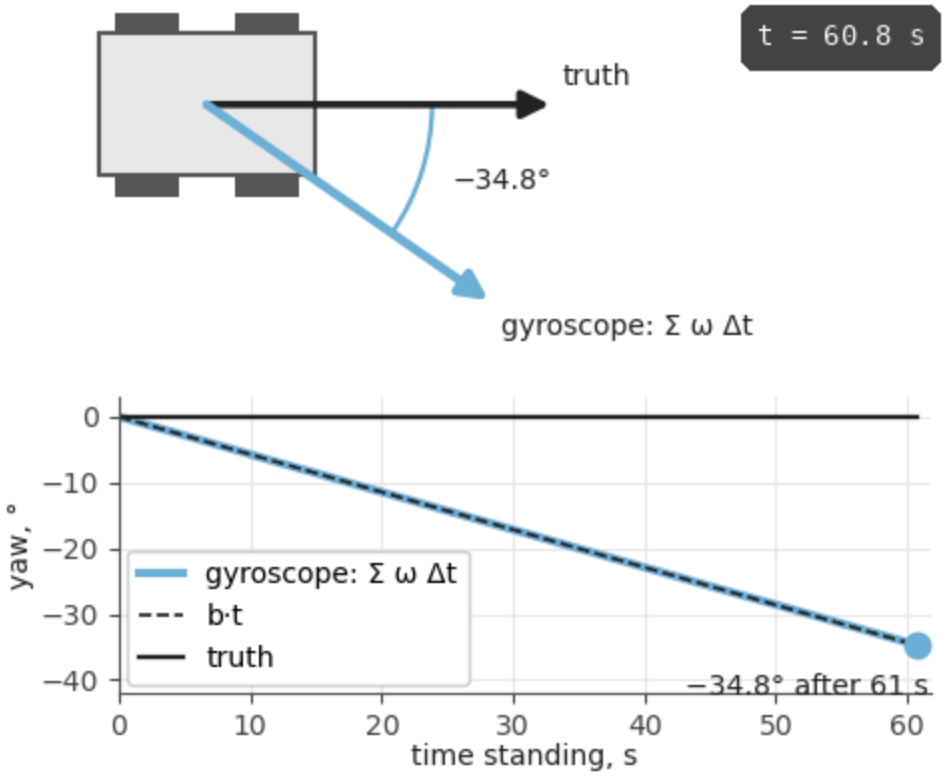


left, RViz ×3: white — the truth, blue — the wheels, green — the EKF on wheels and gyroscope, red — the robot at that EKF's pose · right — the yaw of the same run: blue — the wheels, black — the truth; the cursor at the frame's time

- Four wheels cannot all roll in a turn: they **slip**. Our wheels count -180° per U-turn; the robot truly turns -135° to -142°, **21–25 % less**. Gazebo's Clearpath Husky: about 17 % less
- After two loops the wheels' yaw is 165° off and their position 2.1 m — yet the EKF on the wheels alone claims ±5°: its covariance P knows only the noise it is told

🎯 github.com/gazebosim/gz-sim/issues/872

THE GYROSCOPE DRIFTS EVEN WHEN THE ROBOT STANDS



	WHEELS	GYROSCOPE
what they give	the yaw from wheel turns	the turn rate ω
how they err	slip in turns	the same bias in every reading
when	at once, in each turn	slowly, always — even standing
the error grows with	the number of turns	time
what helps	the gyroscope	the bias, measured standing

- Standing 61 s, the gyroscope's yaw turns **−34.8°** $\approx b\,t$ = -0.0100 rad/s × 60.8 s
- Two kinds of error — the case for fusing them

LEGEND

black arrow and line — the true heading; light blue — the gyroscope's yaw; dashed — $b\,t$; the bold dot — after 61 s

ω — the gyroscope's z reading, one of its three: the turn rate about the vertical, rad/s

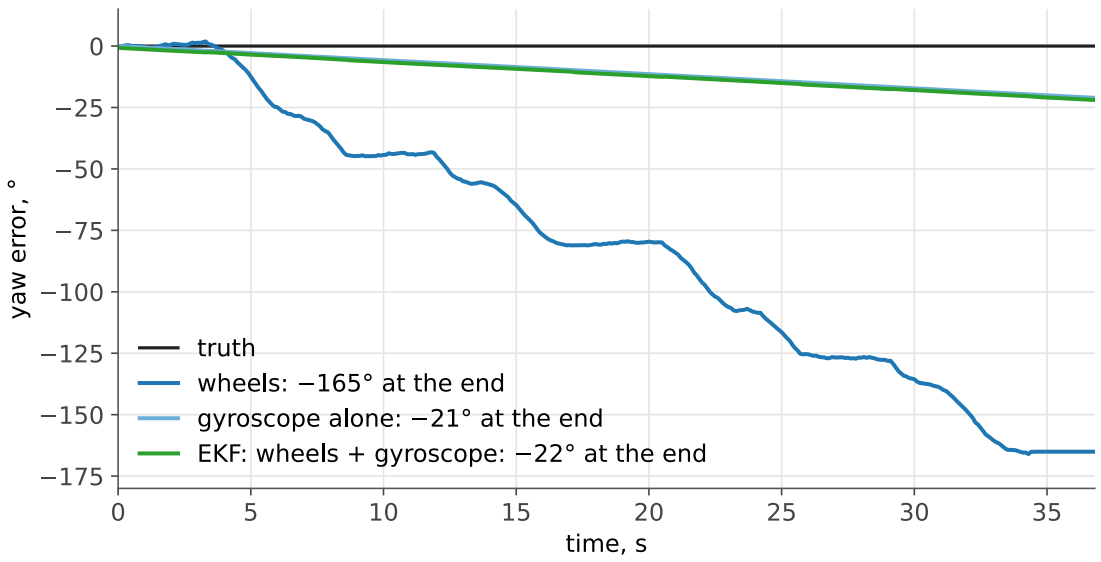
$\Sigma\,\omega\,\Delta t$ — the yaw it adds up from its readings; Δt — the step between readings

b — the gyroscope's bias: the same small error in every reading; our IMU: 0.01 rad/s set, the sign random, measured -0.0100 rad/s

t — time standing, s

$b\,t$ — the yaw error after a time t

FUSION HELPS, BUT THE GYROSCOPE'S BIAS STAYS



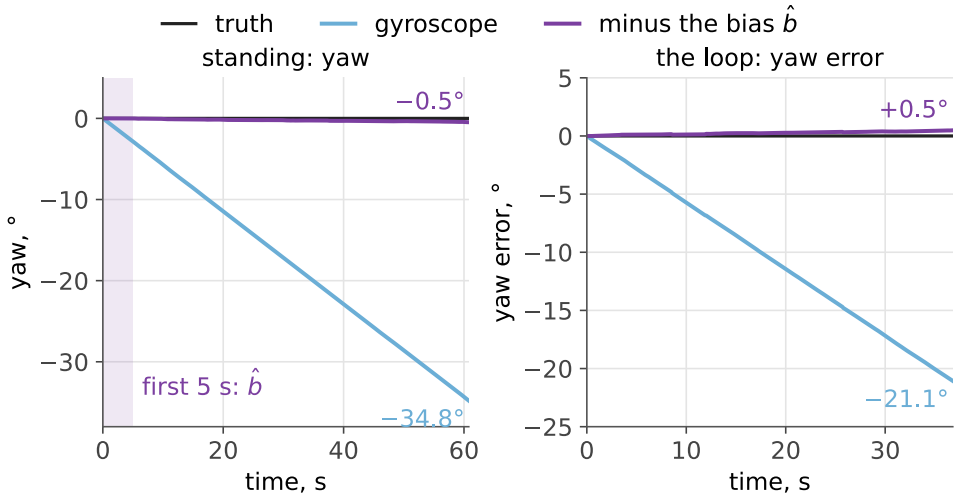
	SPREAD	BIAS
what it is	readings scatter around their mean	the mean is off the truth
on a target	a loose cluster of shots	a tight cluster off the centre
the covariance knows it	yes: σ^2	no
averaging removes it	yes: $\sigma/\sqrt{n}$	no
our gyroscope	σ 0.002 rad/s (the wheels: 0.1)	0.01 rad/s — five times the spread

- After two loops: yaw — wheels **−165°**, gyroscope **−21°**, the EKF on both **−22°**; position — 2.12 m off on the wheels, 0.47 m fused
- The fused yaw follows the gyroscope: its spread is far smaller, so the filter trusts it — bias and all: it claims ±0.4° and is 22° off; standing 61 s, it turns -34.7°, as the gyroscope alone
- Next: measure the bias while the robot stands, and subtract it

MEASURE THE GYROSCOPE'S BIAS WHILE THE ROBOT STANDS

$\hat{b}$ = the mean of ω while the robot stands $\omega - \hat{b}$ — every reading after that

Standing, the true turn rate is 0, so what the gyroscope shows is its bias.



C++ · complementary_filter.cpp — imu_tools: the bias estimate inside complementary_filter_node

```
// standing still?
steady_state_ = checkState(ax, ay, az, wx, wy, wz);
if (steady_state_)
{
    ...
    wz_bias_ += bias_alpha_ * (wz - wz_bias_);
}
...
// every step: the rate minus the bias
double wz_unb = wz - wz_bias_;
```

- Our loop: 5.4 s standing gave $\hat{b}$ = -0.0102 rad/s; subtracted, the gyroscope's yaw after two loops is off by **+0.5°** instead of **−21°**
- `imu_tools` does it by itself: `do_bias_estimation: true` (the default; we switched it off earlier), and `/imu/data` carries $\omega - \hat{b}$ — give that to `robot_localization`

🎯 github.com/CCNYRoboticsLab/imu_tools

LEGEND

light blue — the gyroscope's yaw as it is; purple — with $\hat{b}$ subtracted; purple strip — the first 5 s, where $\hat{b}$ is measured; black — the truth

ω — the gyroscope's reading: the turn rate, rad/s

$\hat{b}$ — its bias, measured: the same small error in every reading

$\omega - \hat{b}$ — the reading without the bias

`checkState`, `steady_state_` — the node's test that the robot stands

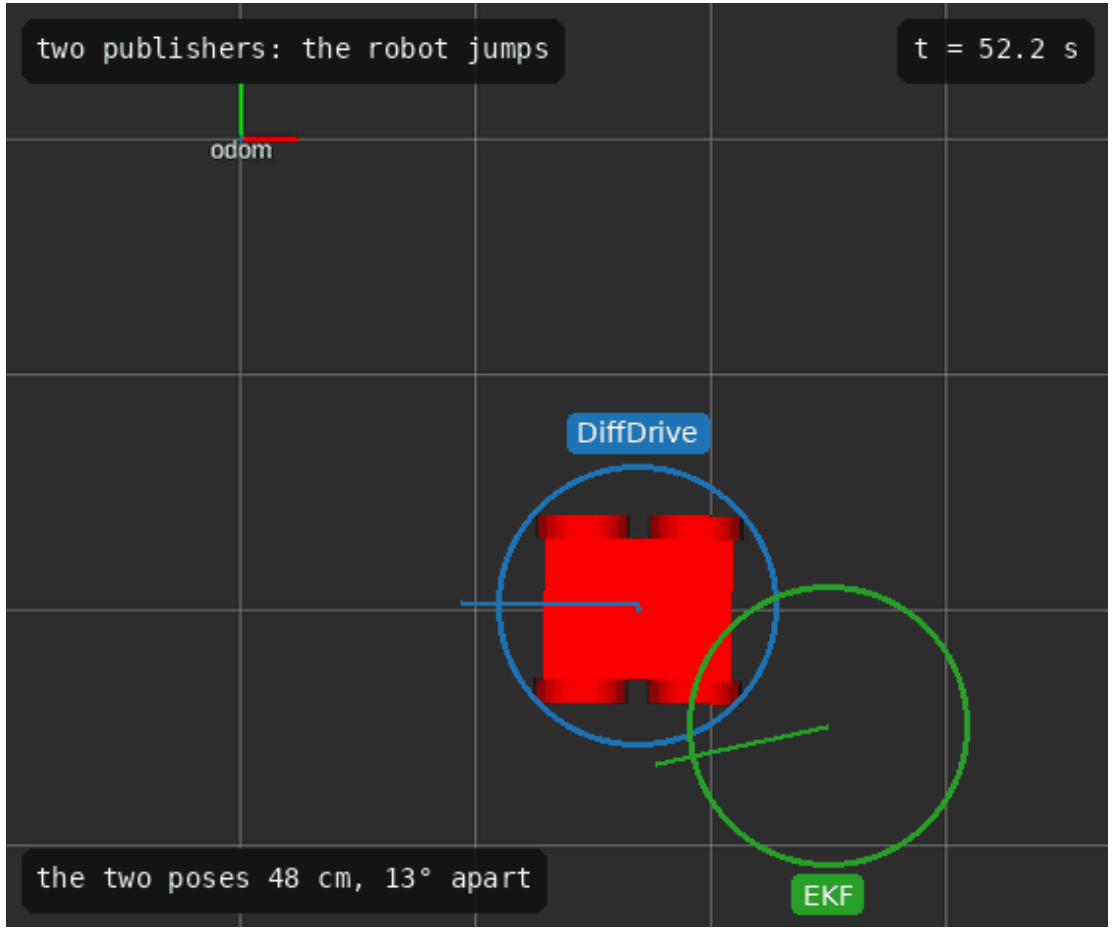
`ax` … `wz` — the IMU's readings; `wz` — ω about the vertical

`wz_bias_` — $\hat{b}$, a running mean; `bias_alpha_` — a new reading's weight in it, 0.01

`wz_unb` — $\omega - \hat{b}$

THE EKF RUNS, AND THE ROBOT IN RVIZ JUMPS BETWEEN TWO PLACES. WHY?

ANSWER: TWO NODES PUBLISH ONE TF LINK




```
$ ros2 topic info /tf --verbose
Publisher count: 3

Node name: robot_state_publisher
...
Node name: ekf_filter_node
...
Node name: ros_gz_bridge
...
```

Two of the three — the EKF (`ekf_node`, running as `ekf_filter_node`) and the bridge with DiffDrive's TF — send the same `odom → base_link`.

- Every TF link has **one** publisher: with two, the robot jumped 24 times a second, by up to 48 cm and 38°
- With `ekf_node` publishing `odom → base_link`, stop bridging the wheels' own TF — no jumps since

docs.nav2.org/jazzy/configuration_and_development/first_time_robot_setup_guide/odom/setup_robot_localization/

THE FILTER FAMILY: BELIEF, PREDICT, UPDATE

FILTER	THE BELIEF	PREDICT	UPDATE	READY
complementary	one angle	+ the gyroscope's turn	a fixed share of the gap to the accelerometer	`imu_complementary_filter`
Bayes on a grid	any shape, on cells	move and blur every cell	× the likelihood, rescale	`filterpy.discrete_bayes`
Kalman	one Gaussian	along a straight line	the weight K	`filterpy` `KalmanFilter`
EKF	one Gaussian	along a curve; the doubt through the tangent	the weight K	`robot_localization` `ekf_node`
UKF (unscented Kalman filter)	one Gaussian	along a curve; the doubt through a few points	the weight K	`robot_localization` `ukf_node`
Madgwick	orientation	+ the gyroscope's turn	a gradient step toward the accelerometer	`imu_filter_madgwick`
particles	a cloud of guesses	move each guess, with noise	weigh each by the likelihood, keep the likely	later: SLAM

- **A few numbers** (a centre and its covariance P) — Kalman, EKF, UKF; **many** (cells, particles) — grid, particles
- **Tracking:** we know where we started. **"Where am I on the map?"** needs many guesses — the next lecture's question

📖 porabook.com/book/

Lecture 7 — SLAM

A map from scans · where am I on that map · both at once — the simplest SLAM

📖 porabook.com/book/